



(12) **United States Patent**
Chao et al.

(10) **Patent No.:** **US 12,403,589 B2**
(45) **Date of Patent:** **Sep. 2, 2025**

(54) **SIMULATING PHYSICAL INTERACTIONS
FOR AUTOMATED SYSTEMS**

(71) Applicant: **Nvidia Corporation**, Santa Clara, CA
(US)

(72) Inventors: **Yu-Wei Chao**, Redmond, WA (US); **Yu
Xiang**, Plano, TX (US); **Wei Yang**,
Lake Forest Park, WA (US); **Dieter
Fox**, Seattle, WA (US); **Chris Paxton**,
Pittsburgh, PA (US); **Balakumar
Sundaralingam**, Seattle, WA (US);
Maya Cakmak, Seattle, WA (US)

(73) Assignee: **Nvidia Corporation**, Santa Clara, CA
(US)

(*) Notice: Subject to any disclaimer, the term of this
patent is extended or adjusted under 35
U.S.C. 154(b) by 302 days.

(21) Appl. No.: **18/148,548**

(22) Filed: **Dec. 30, 2022**

(65) **Prior Publication Data**

US 2023/0294276 A1 Sep. 21, 2023

Related U.S. Application Data

(60) Provisional application No. 63/321,753, filed on Mar.
20, 2022.

(51) **Int. Cl.**
B25J 9/16 (2006.01)

(52) **U.S. Cl.**
CPC **B25J 9/1605** (2013.01); **B25J 9/163**
(2013.01); **G05B 2219/39001** (2013.01)

(58) **Field of Classification Search**
CPC **B25J 9/163**; **B25J 9/1671**; **B25J 9/1605**;
G05B 2219/40309; **G05B 2219/39001**;
G05B 2219/40202

See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS

9,321,176	B1 *	4/2016	Sun		B25J 15/0028
10,471,591	B1 *	11/2019	Hinkle		B25J 9/1612
10,913,151	B1	2/2021	Hinkle		
2012/0165979	A1	6/2012	Lim et al.		
2016/0257000	A1 *	9/2016	Guerin		B25J 9/1671
2018/0319015	A1 *	11/2018	Sinyavskiy		G05D 1/0088
2022/0032454	A1	2/2022	Yang et al.		

FOREIGN PATENT DOCUMENTS

DE	102018220569	A1	6/2020
KR	20170138907	A	12/2017

* cited by examiner

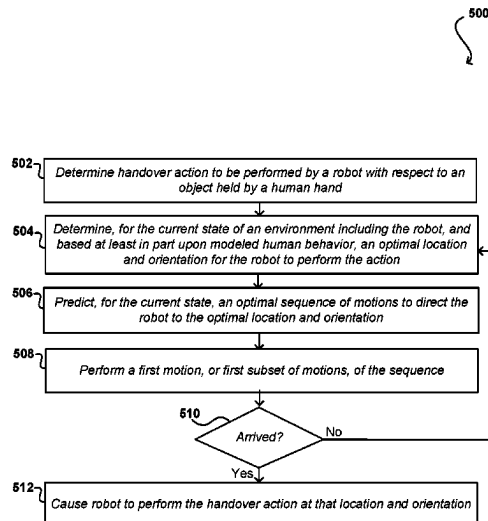
Primary Examiner — Stephen Holwerda

(74) *Attorney, Agent, or Firm* — Hogan Lovells US LLP

(57) **ABSTRACT**

A system may simulate human motion for human-robot interactions, such as may involve a handover of an object. Motion capture can be performed for a hand grasping and moving an object to a location and orientation appropriate for a handover, without a need for a robot to be present or an actual handover to occur. This motion data can be used to separately model the hand and the object for use in a handover simulation, where a component such as a physics engine may be used to ensure realistic modeling of the motion or behavior. During a simulation, a robot control model or algorithm can predict an optimal location and orientation to grasp an object, and an optimal path to move to that location and orientation, using a control model or algorithm trained, based at least in part, using the motion models for the hand and object.

20 Claims, 18 Drawing Sheets



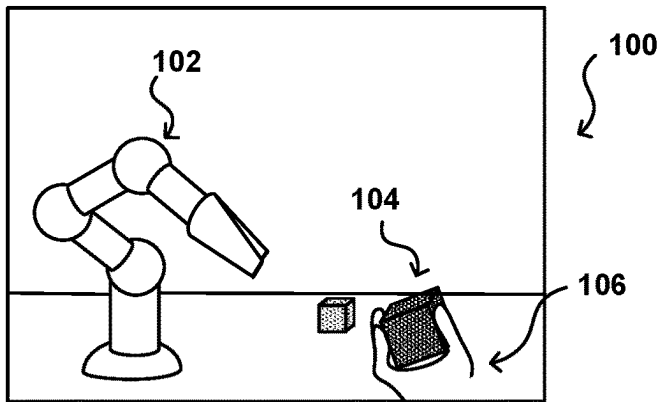


FIG. 1A

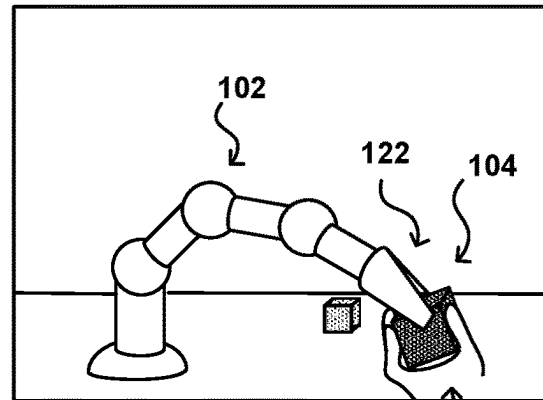


FIG. 1B

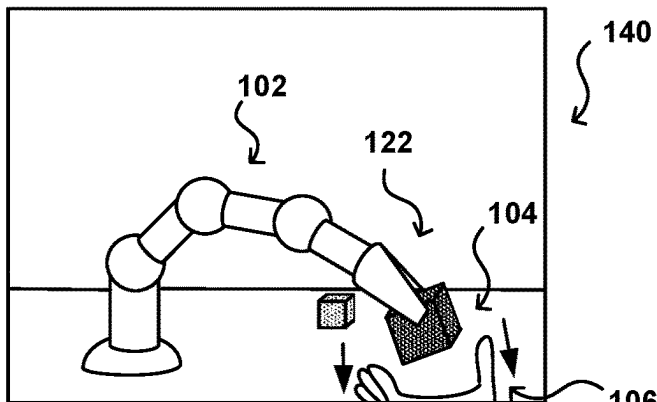


FIG. 1C

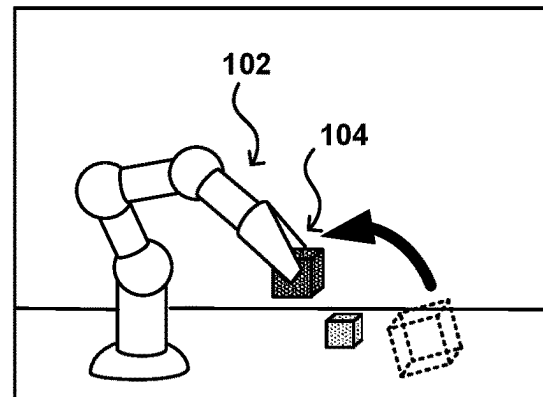


FIG. 1D

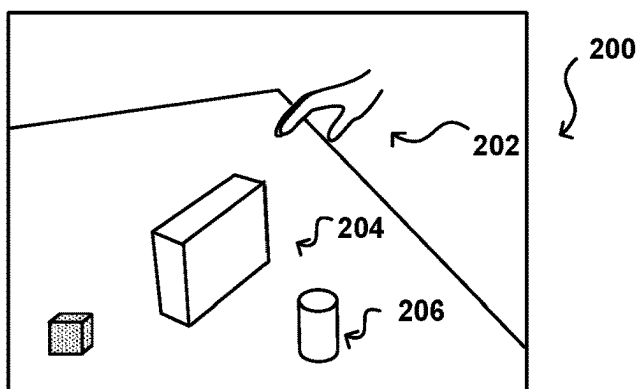


FIG. 2A

220

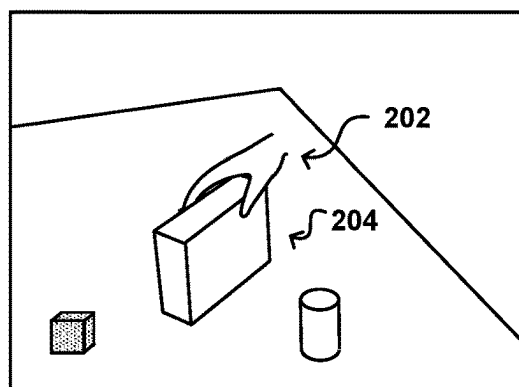


FIG. 2B

240

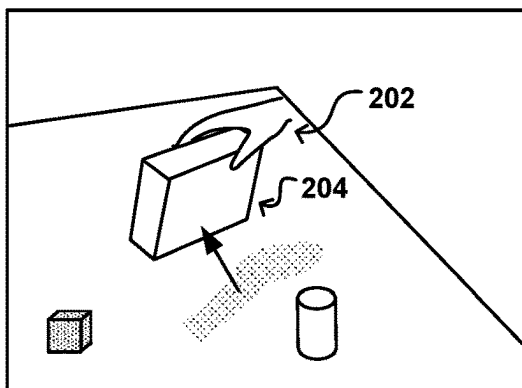


FIG. 2C

260

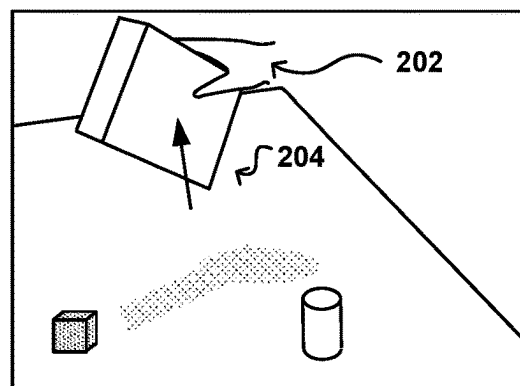


FIG. 2D

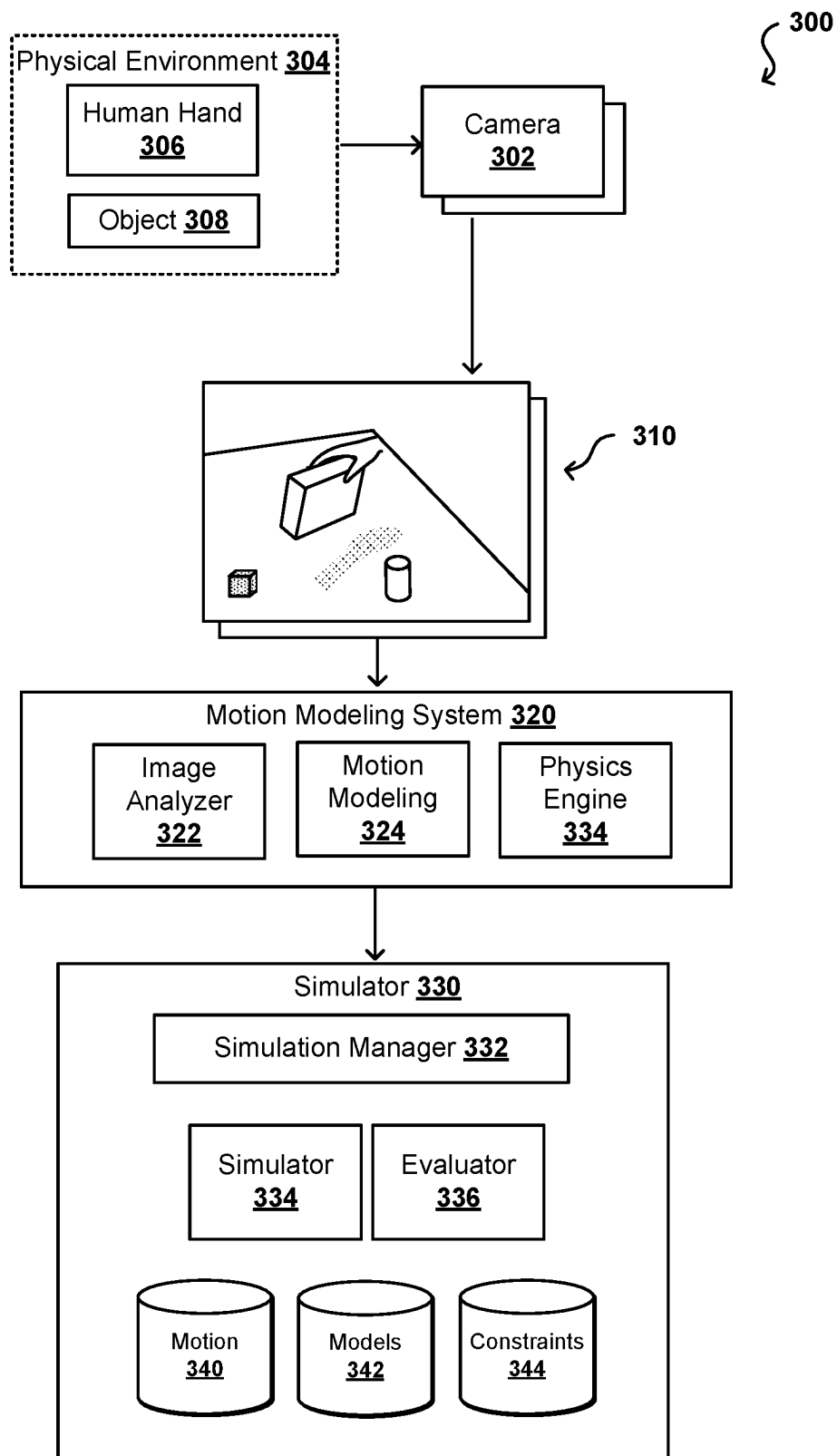


FIG. 3A

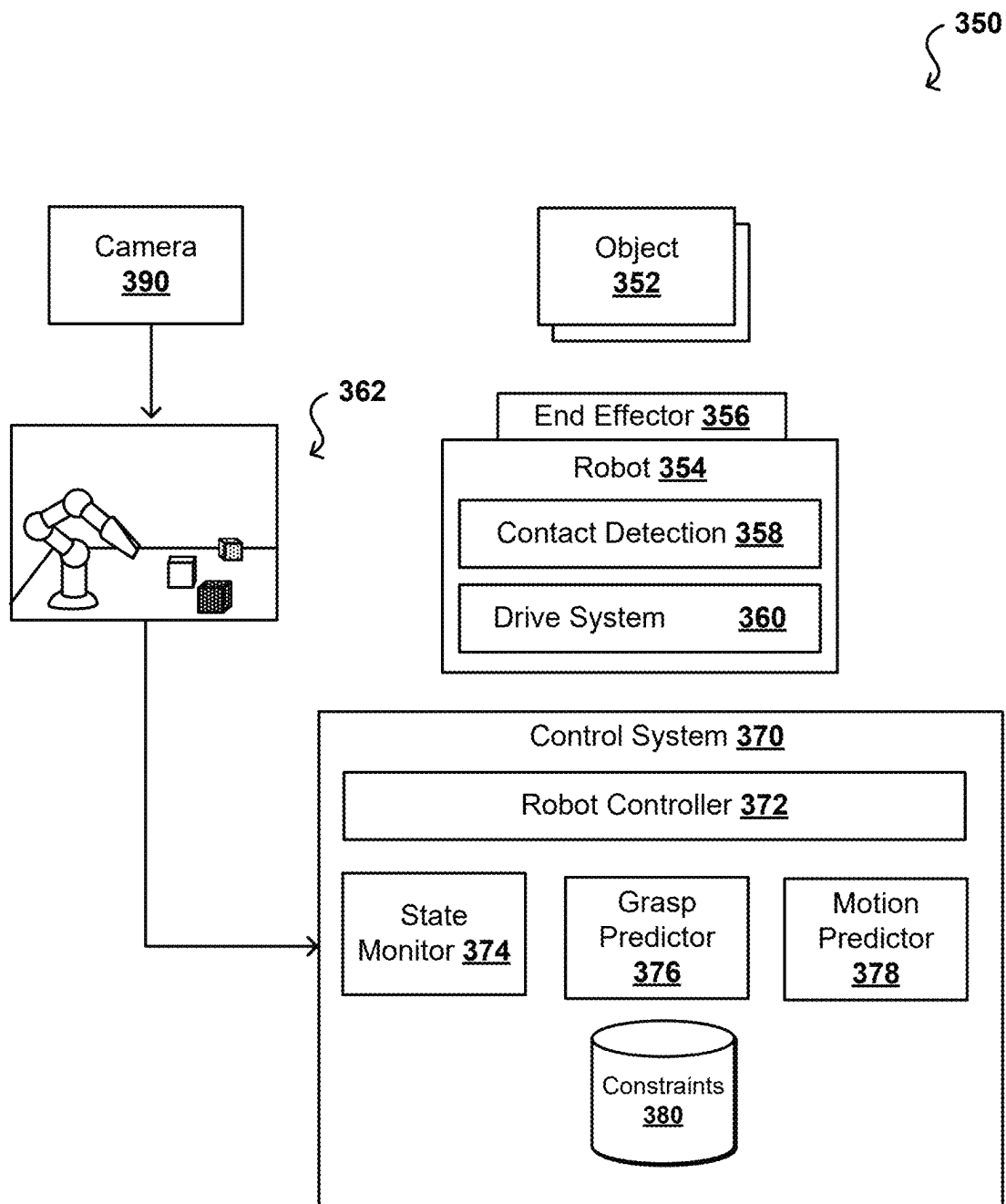


FIG. 3B

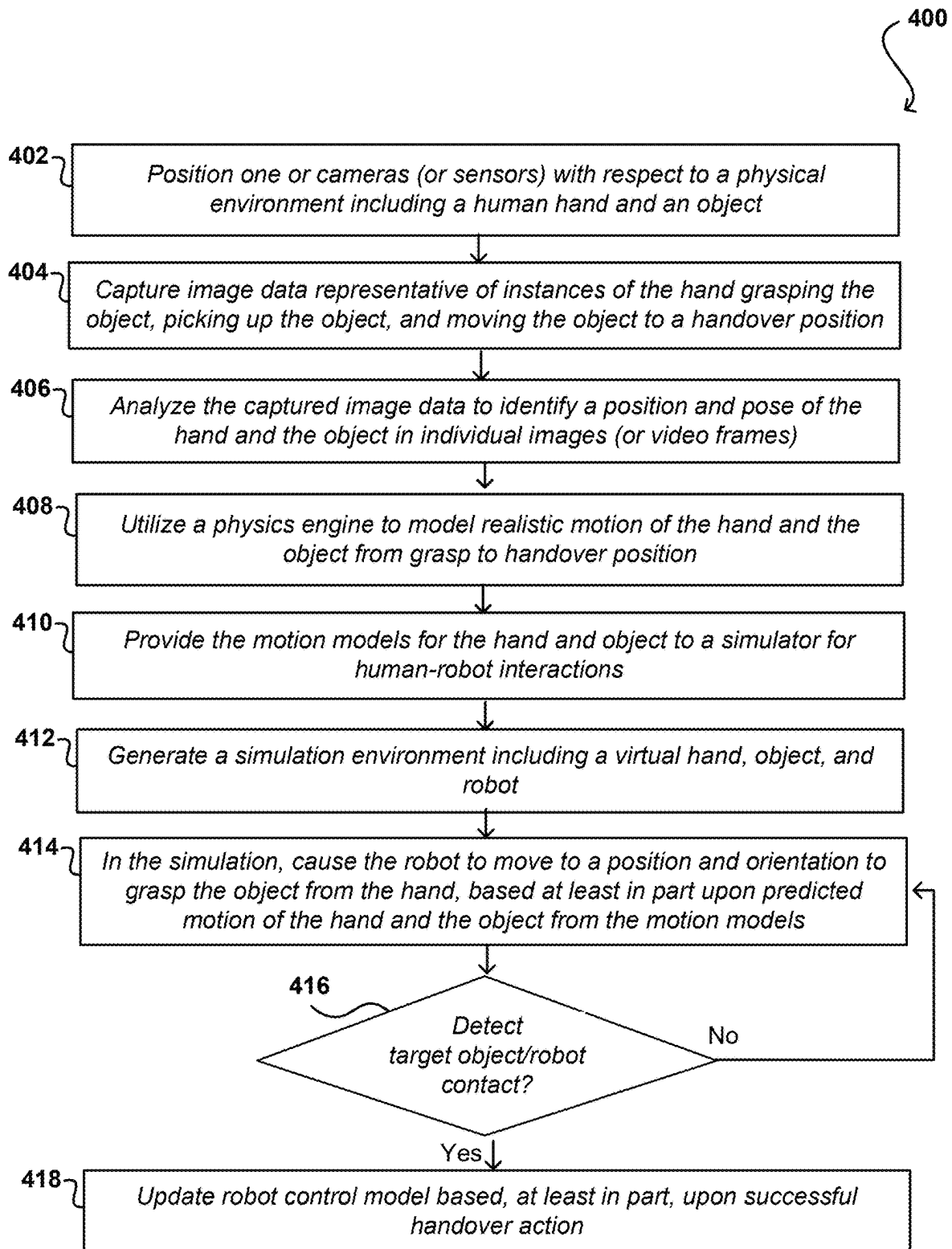


FIG. 4

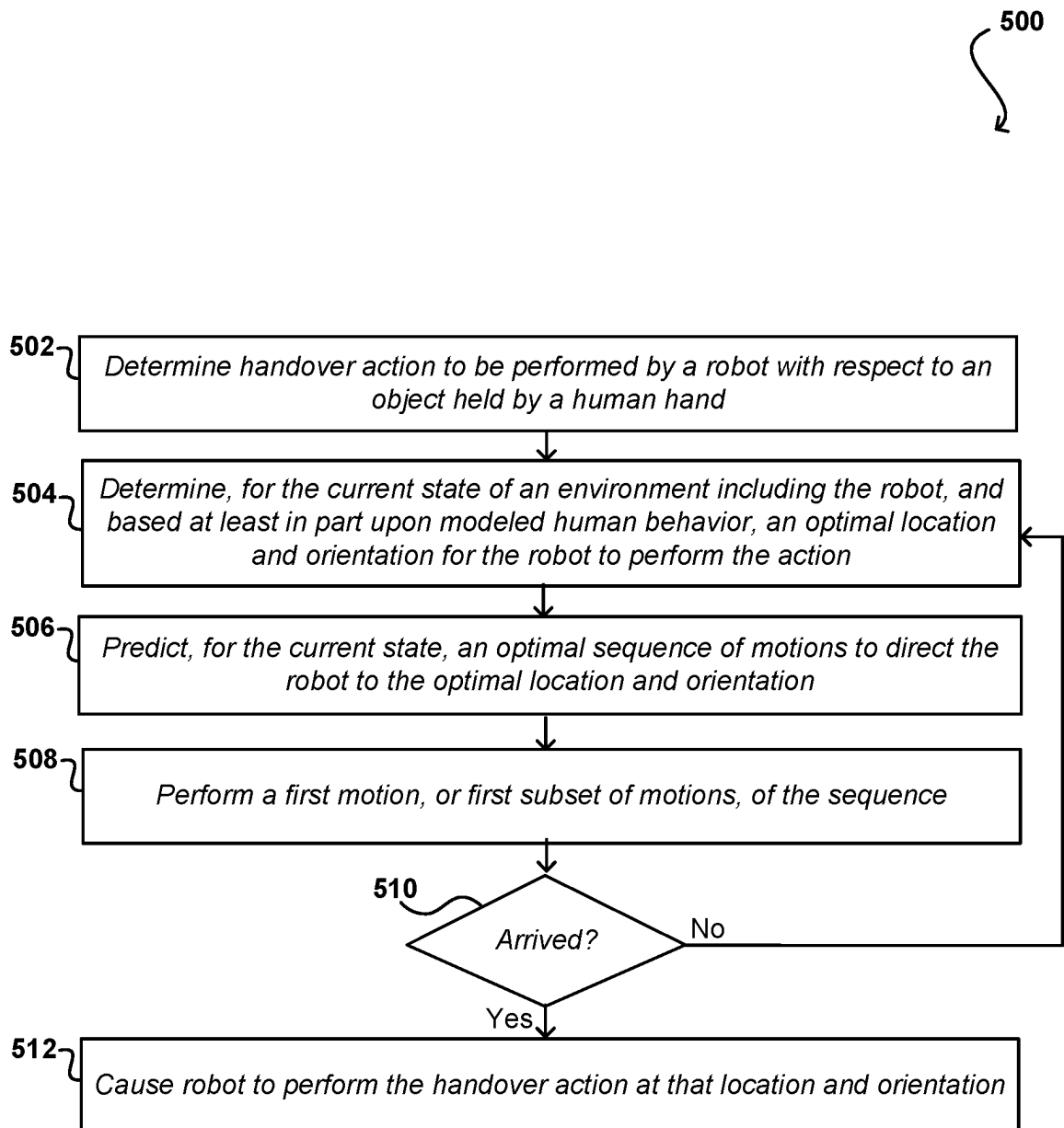


FIG. 5

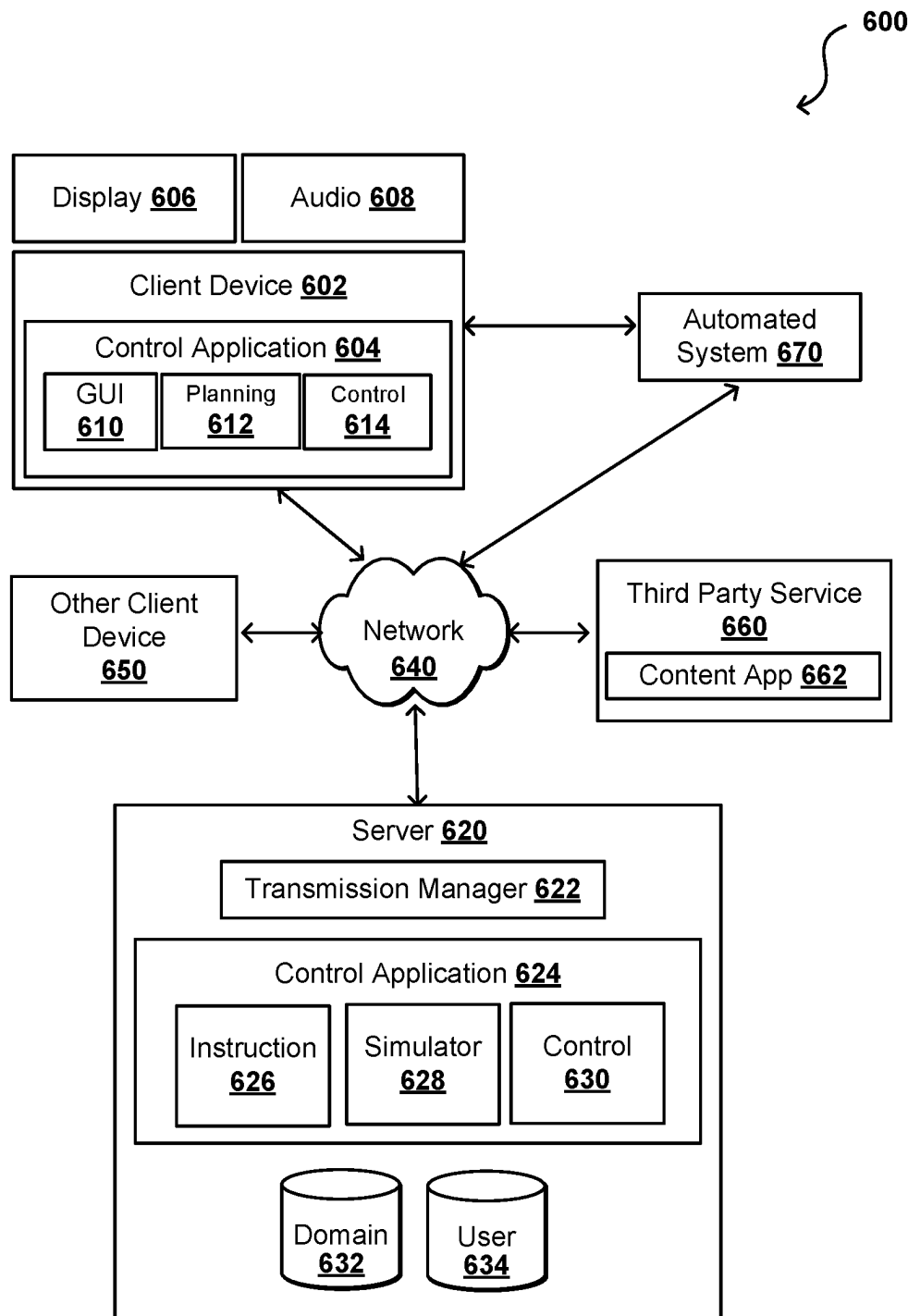


FIG. 6

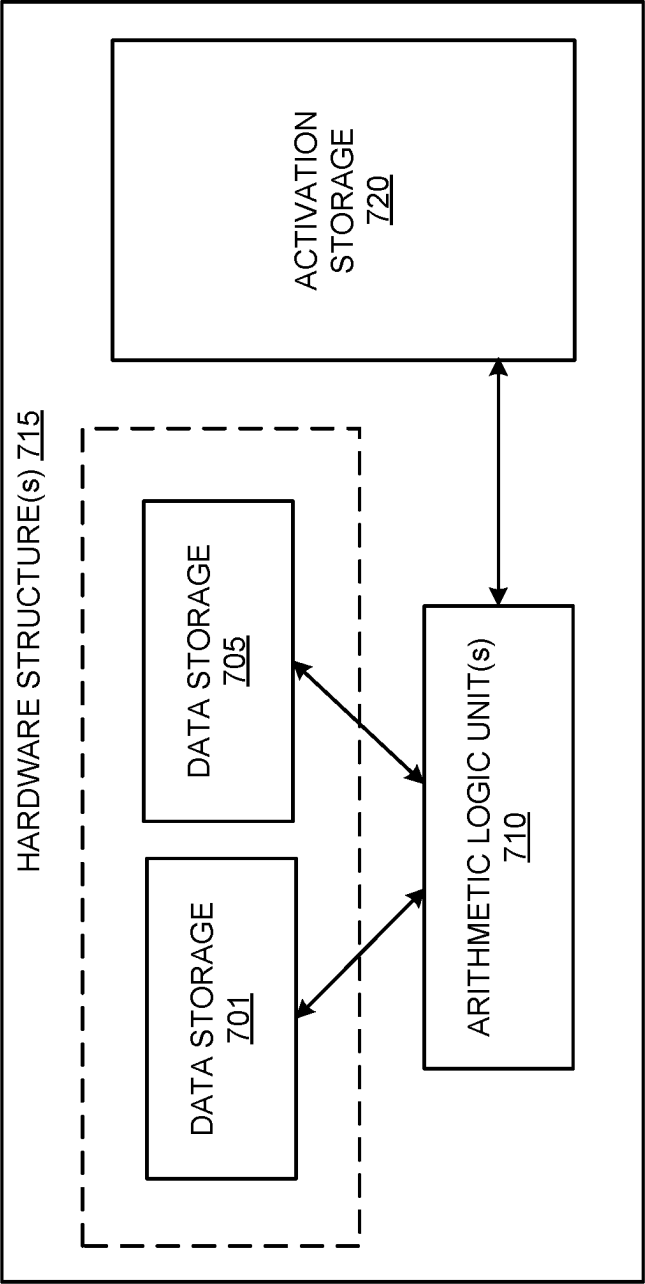


FIG. 7A

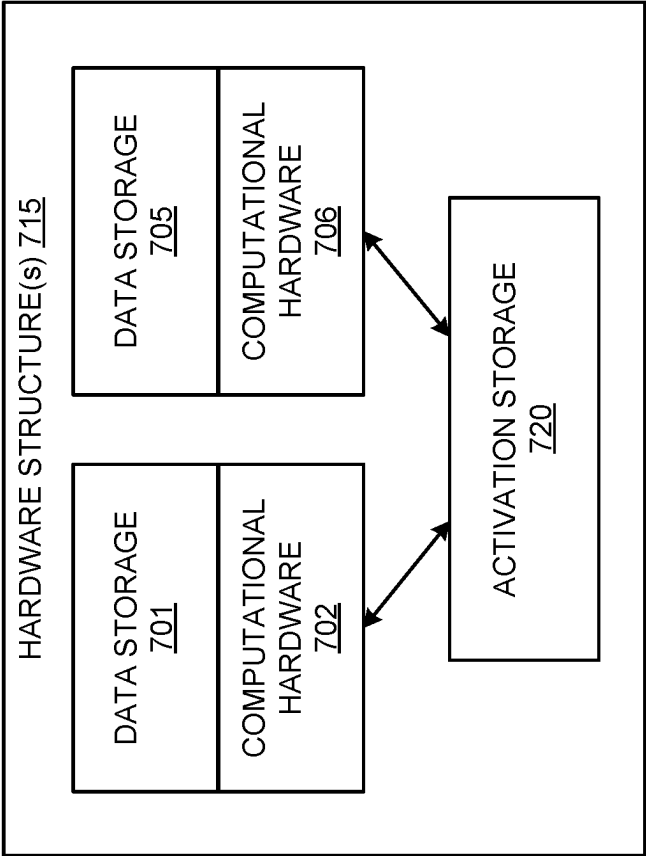


FIG. 7B

DATA CENTER
800

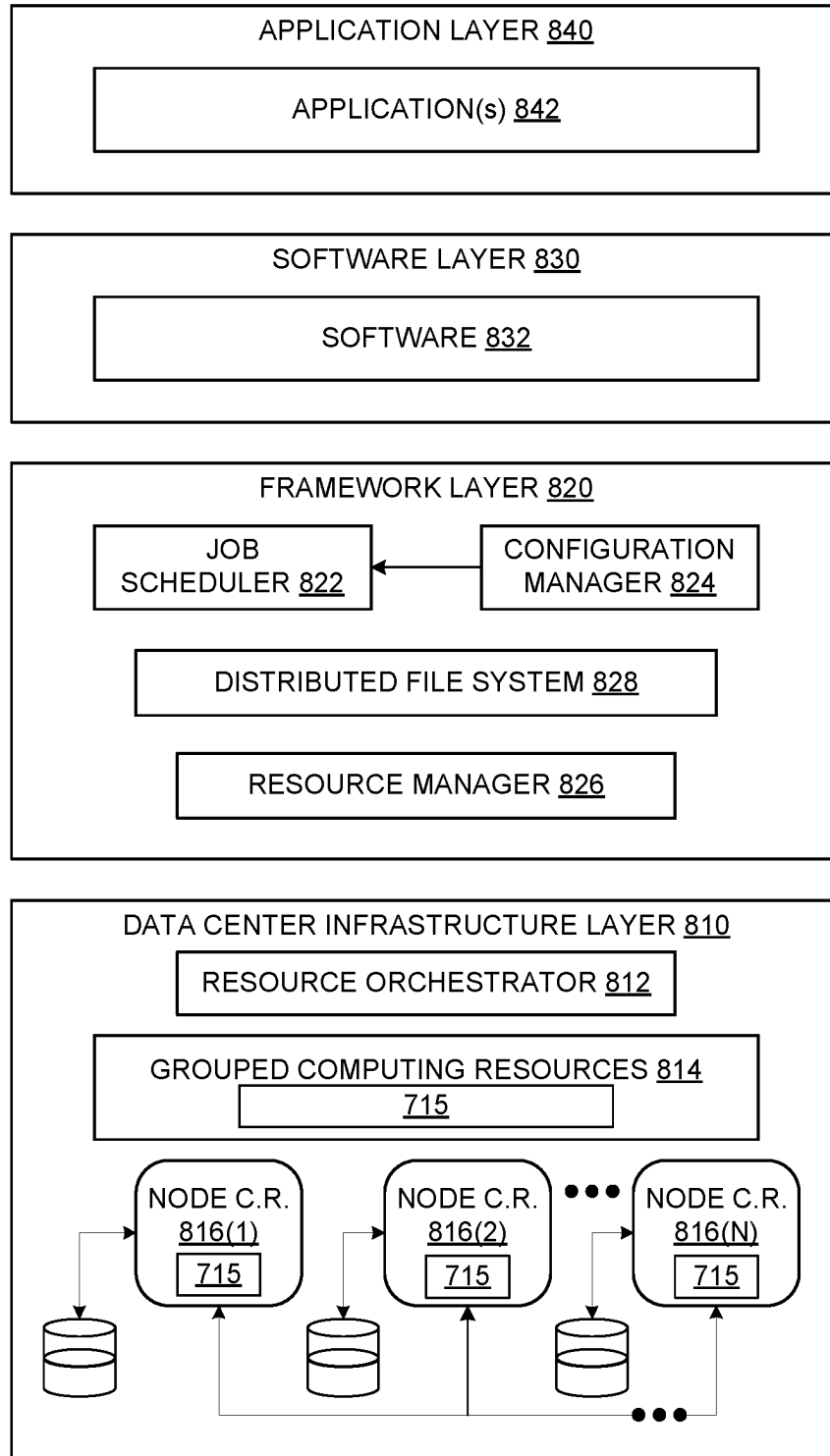


FIG. 8

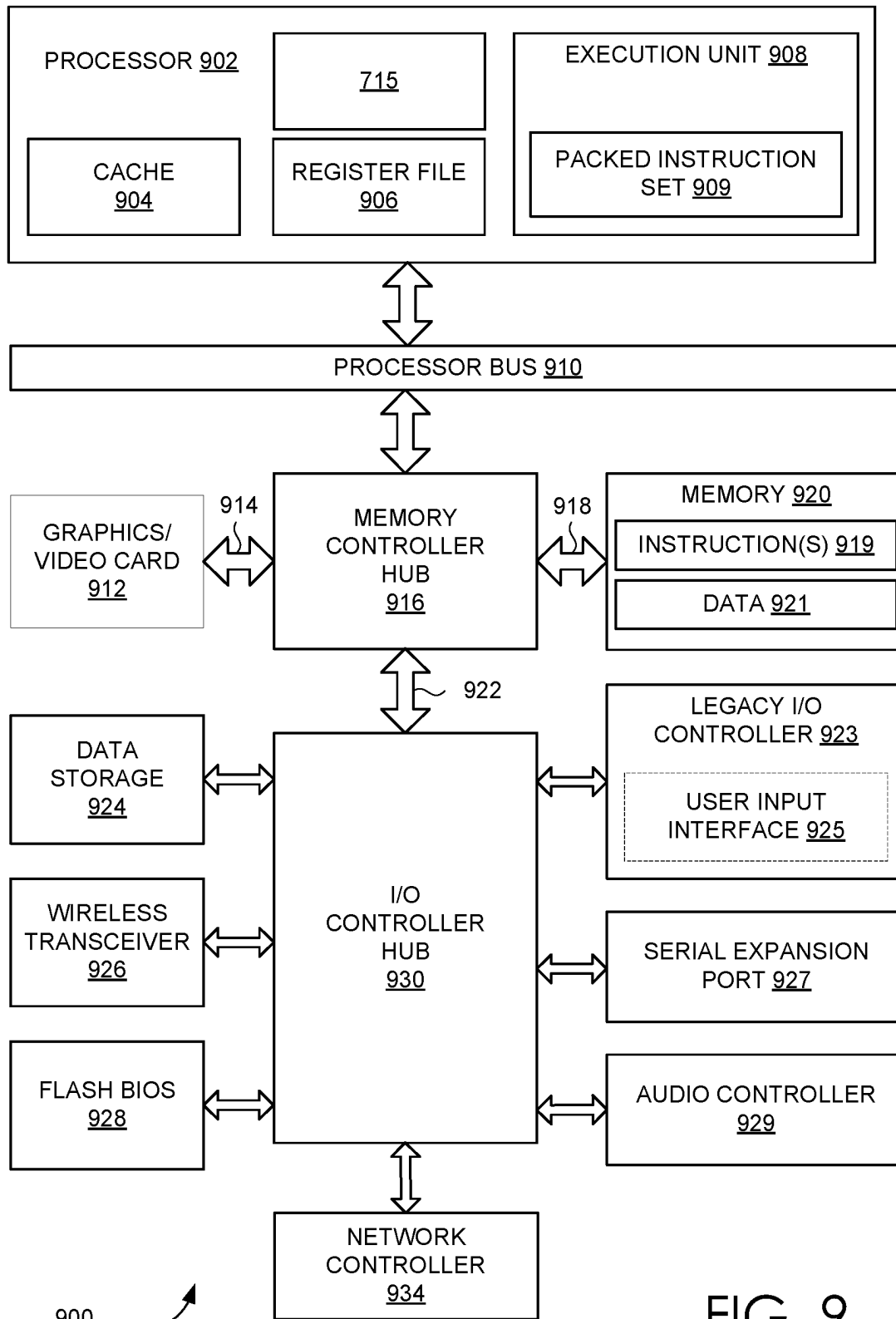


FIG. 9

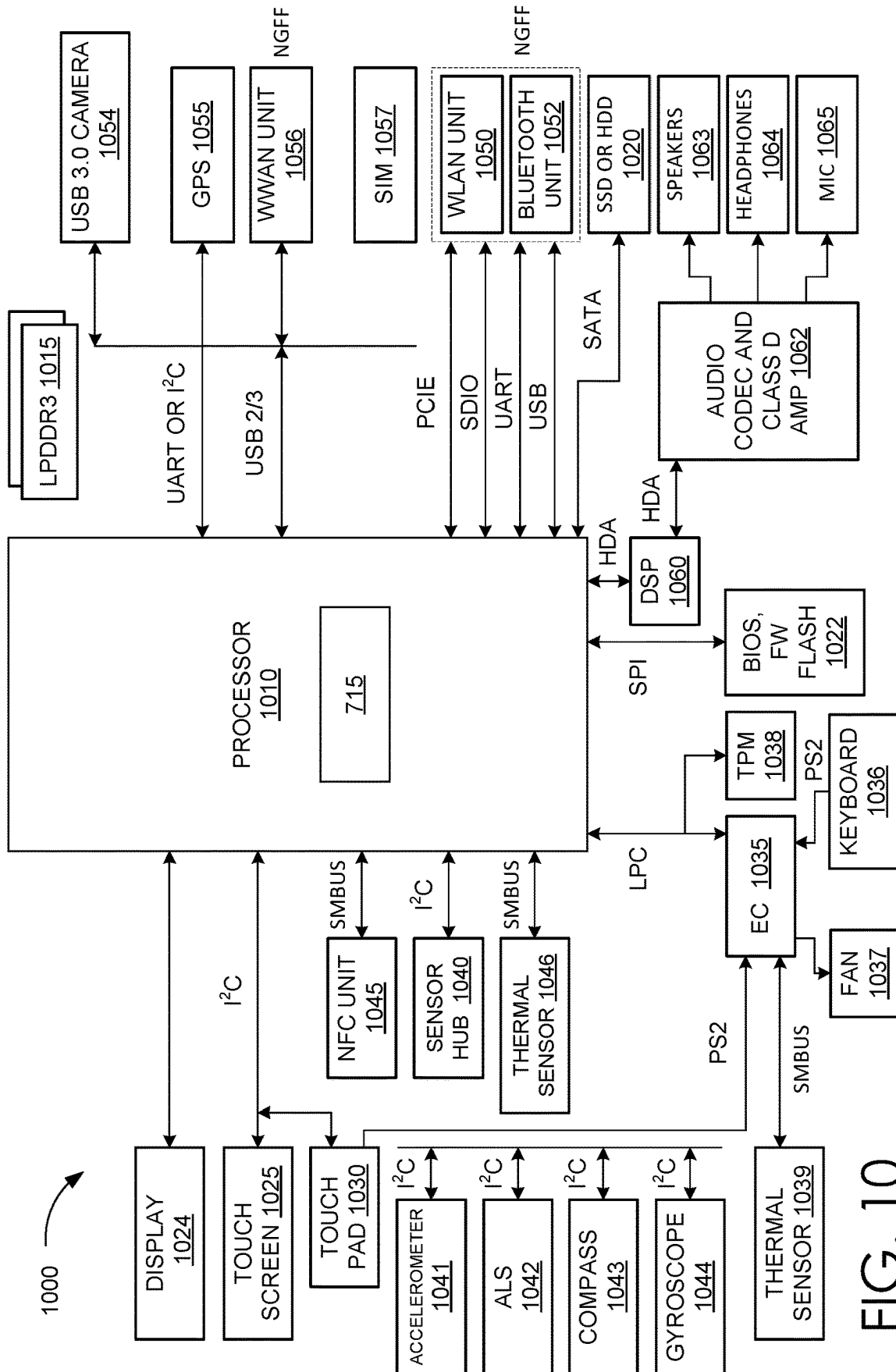


FIG. 10

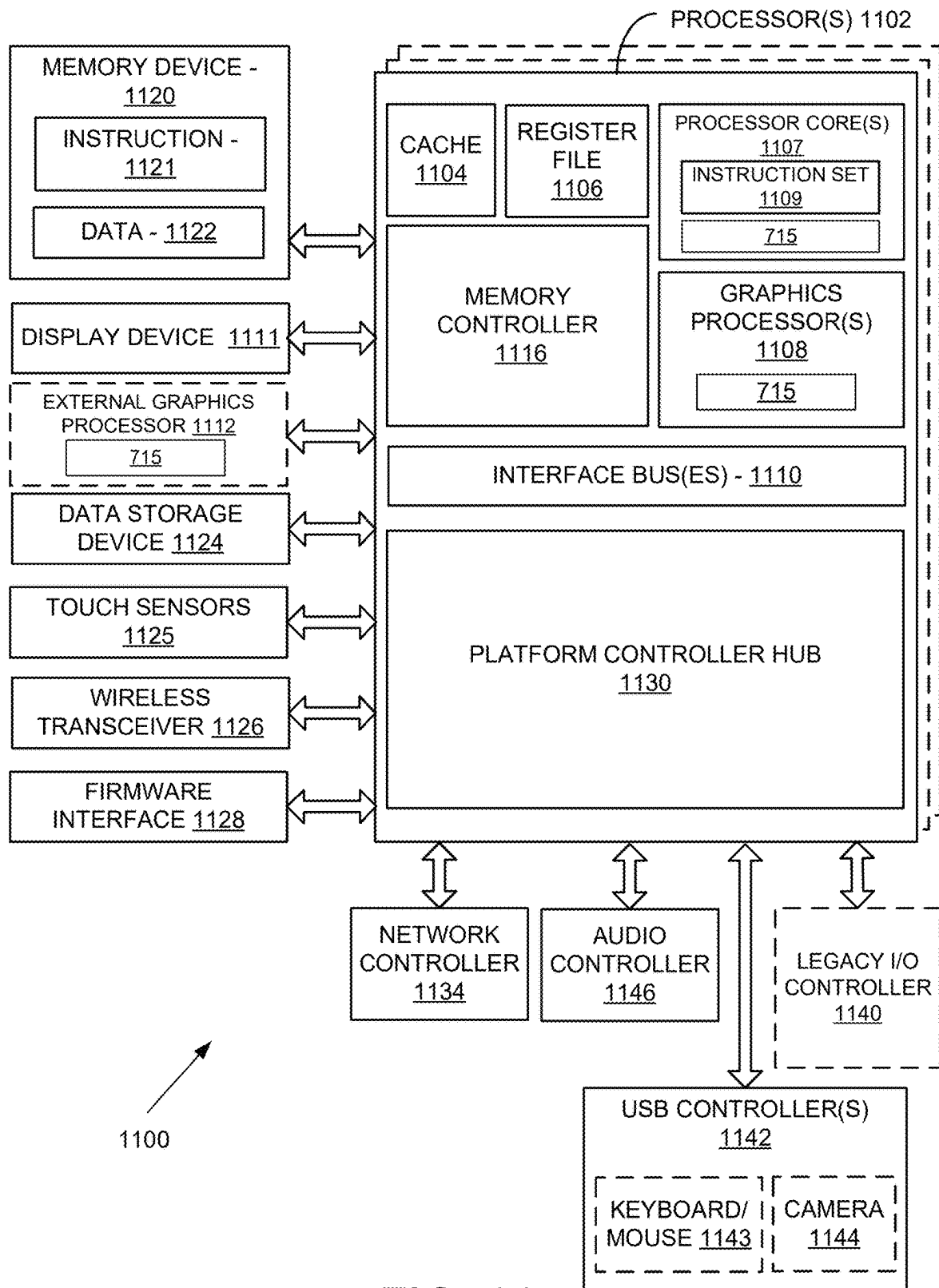


FIG. 11

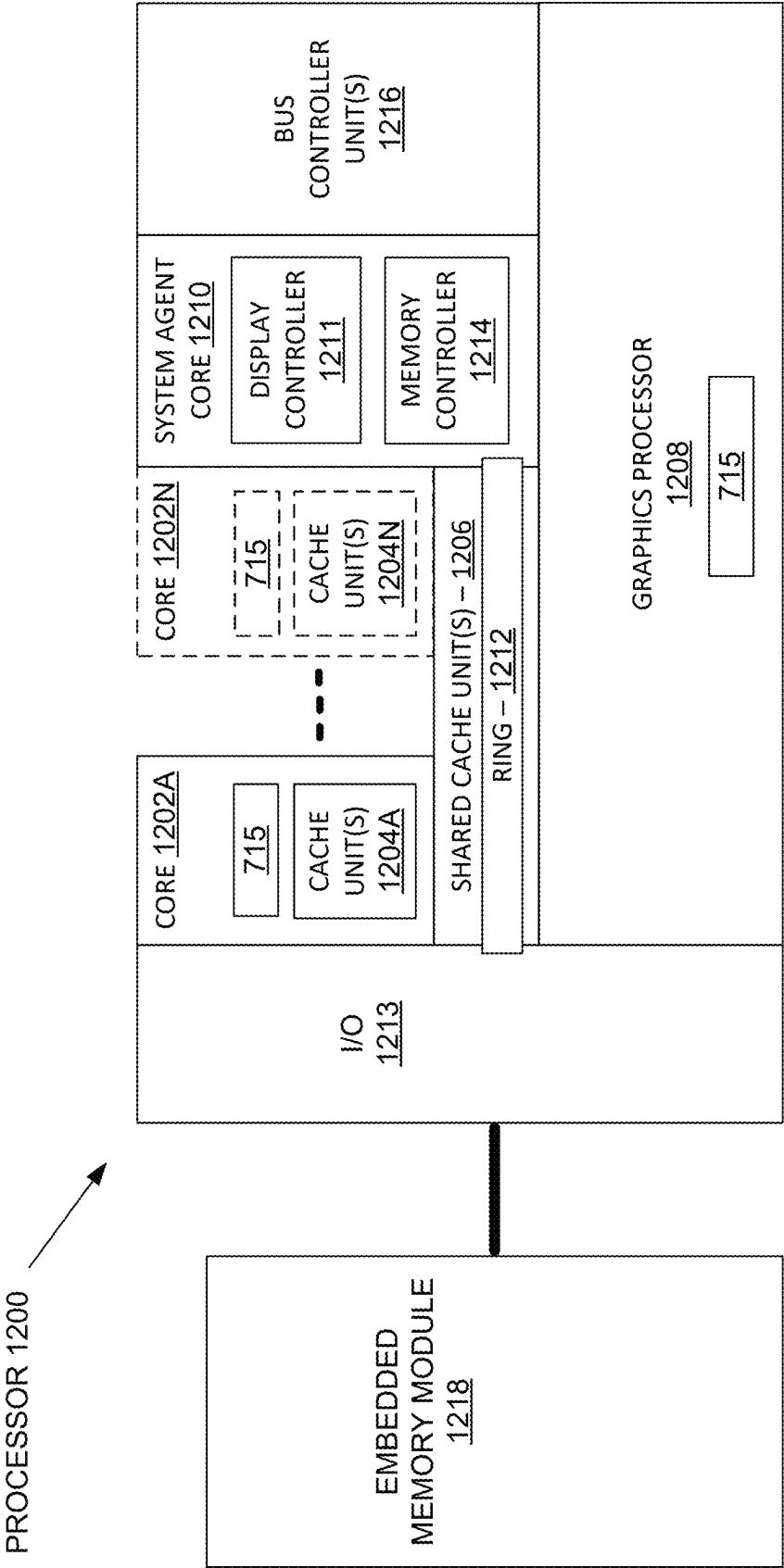


FIG. 12

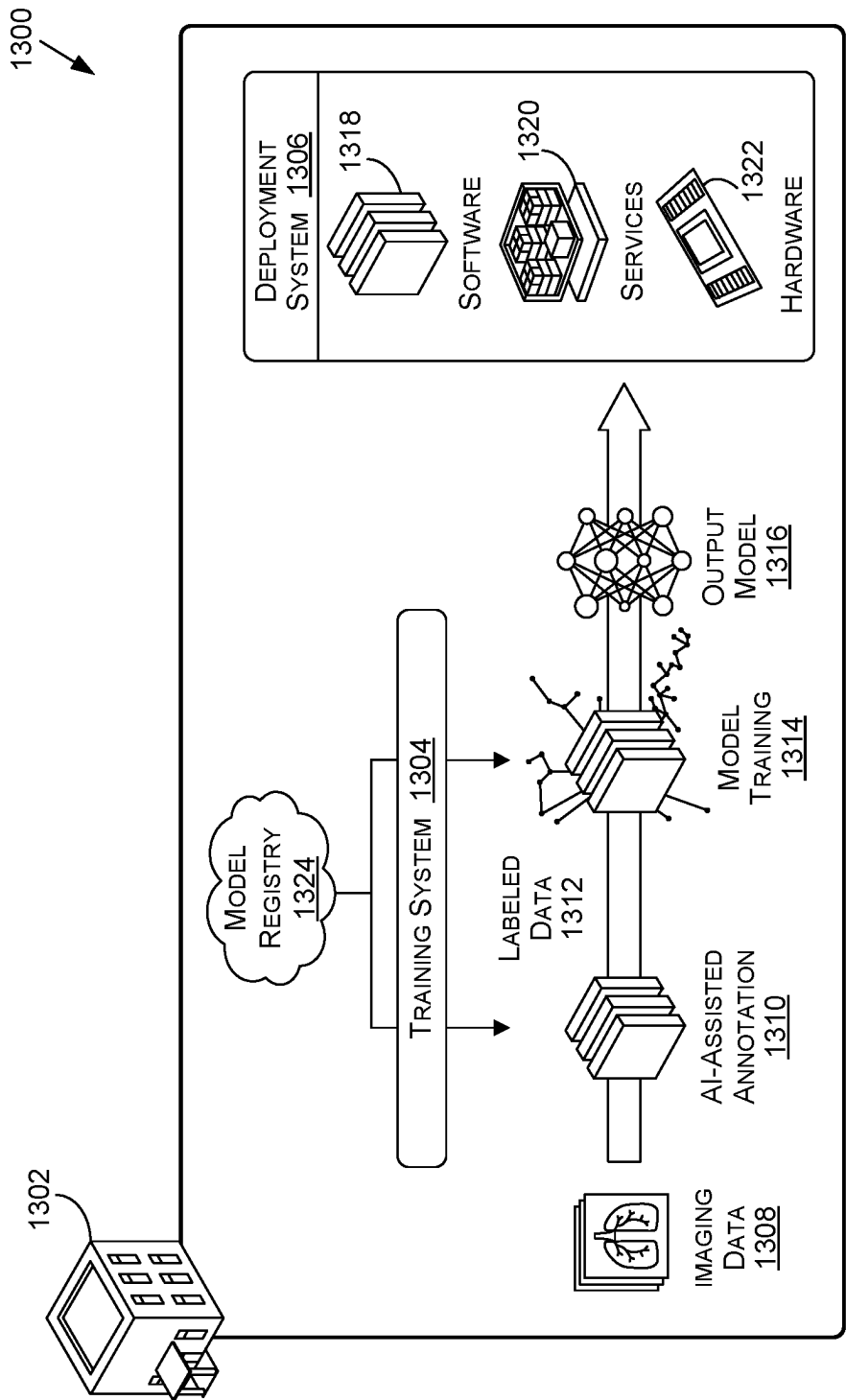


FIG. 13

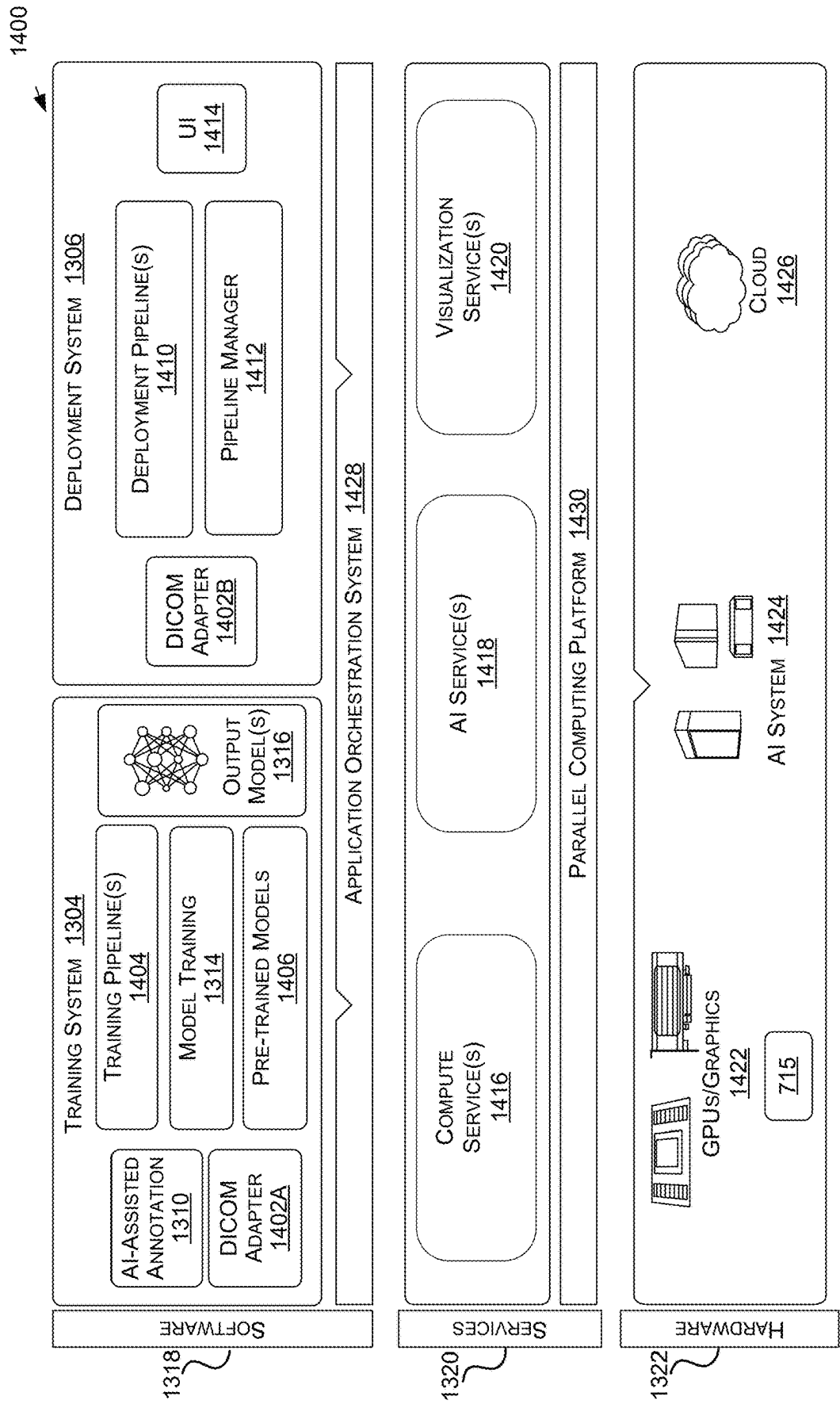


FIG. 14

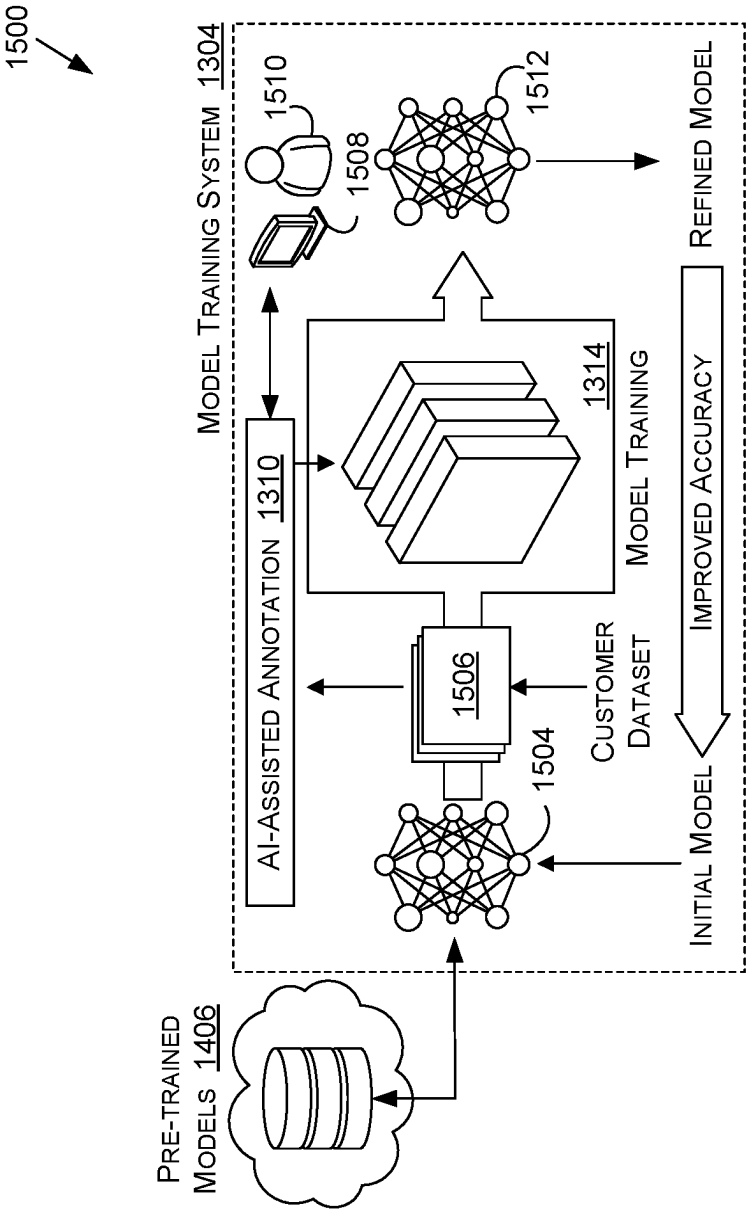


FIG. 15A

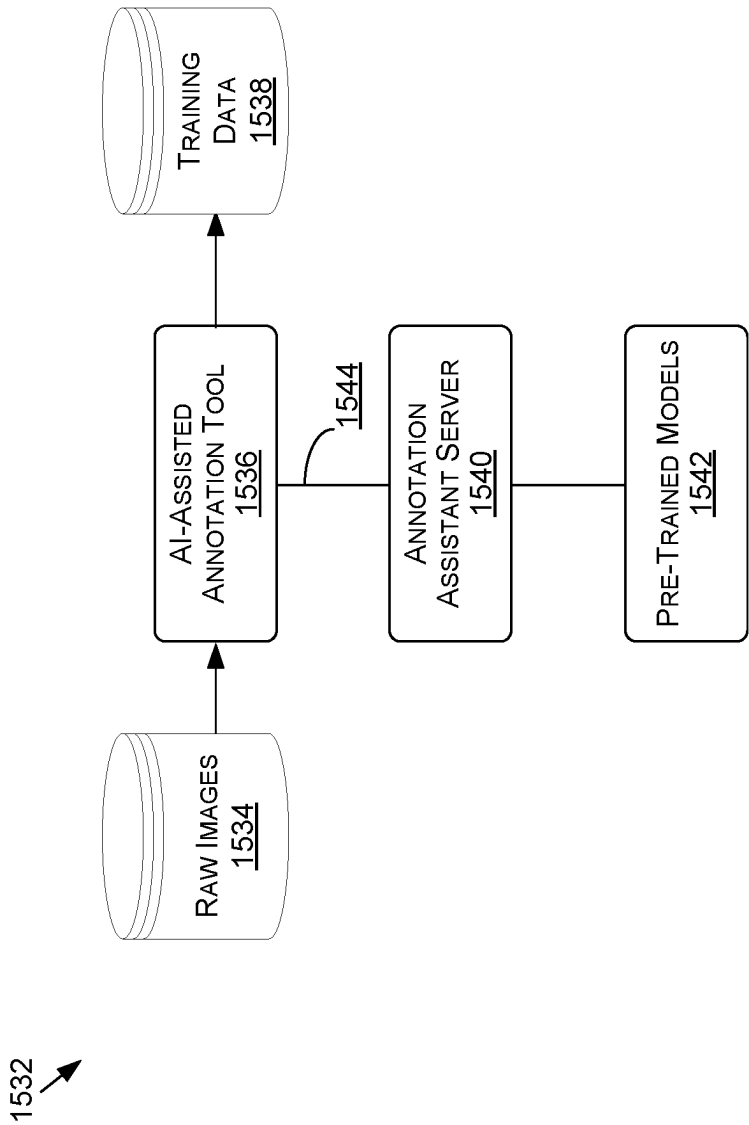


FIG. 15B

SIMULATING PHYSICAL INTERACTIONS FOR AUTOMATED SYSTEMS

CROSS-REFERENCE TO RELATED APPLICATIONS

This application claims priority to U.S. Provisional Patent Application Ser. No. 63/321,753, filed Mar. 20, 2022, and entitled Simulation for Human-Robot Handovers, which is hereby incorporated herein in its entirety and for all purposes.

BACKGROUND

Robots and other automated devices are increasingly being used to assist in the performance of various tasks. At least some of these tasks involve interacting with a human or other entity that requires a substantial degree of dexterity, such as to perform a handover action where the robot is to grasp and take an object from the hand of a person. The ability to exchange objects with humans seamlessly and safely is crucial for human-robot interactions (HRI). Despite increasing efforts in recent years, current development for on human-robot object handovers still faces various challenges. For example, development and testing often requires a real human in the loop, which makes the process expensive, potentially hazardous, and harder to reproduce. Further, different approaches adopt different experimental settings with different types of objects used for handovers, and evaluate with different metrics, which makes cross-study comparison and resulting optimization difficult.

BRIEF DESCRIPTION OF THE DRAWINGS

Various embodiments in accordance with the present disclosure will be described with reference to the drawings, in which:

FIGS. 1A, 1B, 1C, and 1D illustrate images of robot performing a handover operation, in accordance with at least one embodiment;

FIGS. 2A, 2B, 2C, and 2D illustrate images of a human hand grasping an object and moving that object to a location and orientation appropriate for a handover action, in accordance with at least one embodiment;

FIGS. 3A and 3B illustrate example systems for simulating human behavior, training a robot control model based on that simulated behavior, and performing human-robot interactions using such a robot control model, according to at least one embodiment;

FIG. 4 illustrates an example process for simulating human motion for use in training a robot control model, according to at least one embodiment;

FIG. 5 illustrates an example process for causing a robot, or automated assembly, to move to a determined location and orientation to perform a task, according to at least one embodiment;

FIG. 6 illustrates components of a distributed system that can be used to model human behavior, train a robot control model using that behavior, and/or cause a robot to perform one or more tasks using that robot control model, according to at least one embodiment;

FIG. 7A illustrates inference and/or training logic, according to at least one embodiment;

FIG. 7B illustrates inference and/or training logic, according to at least one embodiment;

FIG. 8 illustrates an example data center system, according to at least one embodiment;

FIG. 9 illustrates a computer system, according to at least one embodiment;

FIG. 10 illustrates a computer system, according to at least one embodiment;

FIG. 11 illustrates at least portions of a graphics processor, according to one or more embodiments;

FIG. 12 illustrates at least portions of a graphics processor, according to one or more embodiments;

FIG. 13 is an example data flow diagram for an advanced computing pipeline, in accordance with at least one embodiment;

FIG. 14 is a system diagram for an example system for training, adapting, instantiating and deploying machine learning models in an advanced computing pipeline, in accordance with at least one embodiment; and

FIGS. 15A and 15B illustrate a data flow diagram for a process to train a machine learning model, as well as client-server architecture to enhance annotation tools with pre-trained annotation models, in accordance with at least one embodiment.

DETAILED DESCRIPTION

In the following description, various embodiments will be described. For purposes of explanation, specific configurations and details are set forth in order to provide a thorough understanding of the embodiments. However, it will also be apparent to one skilled in the art that the embodiments may be practiced without the specific details. Furthermore, well-known features may be omitted or simplified in order not to obscure the embodiment being described.

Approaches in accordance with various embodiments can provide environments for simulating actions, such as may correspond to the handover of an object from a human to a robot. Such an environment can be used for training a robot control model or algorithm to perform such handover actions, as well as to evaluate the performance of the robot handovers. In order to simulate realistic human hand motion, motion capture can be performed to obtain a significant amount of hand motion or behavior data. The motion capture does not need to involve a robot or actual handover action, but can include motion and behavior of a hand (or other feature) grasping an object and moving that object to a location and orientation that is appropriate for a handover to occur. Motion data obtained from such a capture process can be fed to (for example and without limitation) a physics engine to simulate realistic motion for both the hand and the object. In one or more embodiments, collision detection can be disabled, constrained, or otherwise limited between the hand and the object, which can help to avoid instabilities in the simulation that may result from noise or variability in the motion capture data. Such a simulation can provide realistic hand and object motion, along with realistic grasping motion of the hand with respect to the object, such that an accurate determination can be made as to where the robot should grab the object in order to avoid pinching the hand of the person, or violating another such constraint. Once the robot has sufficiently contacted or grasped the object in the simulation, any controller for the object (separate from a controller for the hand) can be deactivated or disabled, such that if the robot does not accurately grip the object the object will fall to the ground (due to gravity in the simulated environment).

Various other such functions can be used as well within the scope of the various embodiments as would be apparent to one of ordinary skill in the art in light of the teachings and suggestions contained herein.

FIG. 1A illustrates a first image **100** of an example environment in which a robot **102**, or at least partially automated assembly, can perform various tasks. Such an environment for operating a robot may include any appropriate environment, such as a laboratory, user home, warehouse, workbench, vehicle, hospital or factory, among others, which may be co-located with a user or operator of the robot, or may be at a different geographic location as may be separated by a large distance. While blocks are illustrated as example objects with which a robot can interact, it should be understood that such objects could be any physical objects that could be located in one of these operating environments. This image **100** may be captured by a camera, which may be of a set one or more cameras, that are used to capture image or video data about this environment, to help guide the robot **102** in performing tasks or actions within the environment, such as by using computer vision analysis to determine positions and orientations of the robot **102** and one or more objects **104** for interaction, as well as objects with which to avoid collisions.

In this example, a target block **104** for a human-robot interaction is being held in a human hand **106**. A task to be performed by the robot **102** may include performing a handover task, which can involve causing a gripper **122**, end effector, or other such portion of the robot **102** to perform a grasp action as illustrated in image **120** of FIG. 1B. In addition to grasping the block **104** in a way that does not pinch or otherwise contact the human hand **106**, for example, the gripper can be instructed to hold the block **104** in such a way that the human hand **106** can let go of the block and the block **104** will remain held by the gripper **122** of the robot **102**. Once the block **104** is held securely by the robot and the hand **106** has released the block, and moved at least a secure distance away from the block **104** such as illustrated in image **140** of FIG. 1C, the robot **106** can then perform various actions with respect to the block, or another such object taken during a handover action, such as to move the block **104**, as illustrated in image **160** of FIG. 1D, and set the block down in a target position (or perform another such action).

As mentioned, it can be desirable during a task such as a handover to avoid any contact between the robot and the hand, or other portion, of the person holding the object for the handover action, unless that contact is required for performance of the task. Avoiding contact can prevent pinching or causing any harm to the person, as well as preventing the human from dropping or damaging the object in response to the unexpected contact. It can also be desirable for the robot to move towards the object held by the person in a smooth, reliable, and intuitive manner, such that the person is not startled by the movement or triggered to drop or damage the object as a result of a “jerky” (e.g., sudden change in speed and/or direction) or unexpected movement of the robot. In some embodiments, causing the human to move unexpectedly may not damage the object, but may result in an obstruction of a camera used in guiding the robot or other such undesirable result, or at least may delay the performance of the handover as the robot will need to adjust to the new position of the object.

In at least some embodiments, a robot can be trained to perform handovers and other such actions, or interactions, by learning (through the robot itself or a control module or algorithm) safe and acceptable ways to, for example, approach, grasp, and hold an object in a handover scenario. In order to obtain this learning, a number of handover operations can be performed that can be labeled or scored based upon a determination of successor other such crite-

rior. As mentioned, this can be performed in a physical setting where a robot takes an object from a human, for example, but such an approach may come with a number of deficiencies. For example, such a process may be slow and expensive to set up and operate. Such an approach may also provide for limited training data, unless a large number of handover actions are performed and evaluated in a well-planned way so as to account for a wide variety of possibilities. Such an approach also comes with at least some risk of harm to the person involved, as the robot (or control algorithm) has not yet learned to safely and reliably perform a handover (or other such) action.

Instead of using humans to perform handover actions in a real-world environment, handovers and similar robot-human interactions can be simulated in a virtual testing environment. In such a simulation, a virtual hand can be simulated to hold a virtual object. A virtual robot can also be simulated, which can be instructed to attempt to perform a handover action with respect to the object, given constraints such as to not contact the virtual hand or to not obstruct a virtual camera view. A success score, or other such metric, can be generated or determined for each such simulation, and this feedback can be used to adjust the robot handover control instructions, which may be determined using a neural network, predictive algorithm, or other such approach. An advantage of simulating these experiments is that they can often be repeatedly performed much faster, safer, and cheaper than a physical, real-world experiment with a human. However, the usefulness of a simulation is generally limited to the accuracy and realism of the simulation. For example, a simulation for a handover action may include a virtual hand holding a virtual object, but the way that hand holds the object might not be realistic, or may not include a variety of possible ways different people could hold different objects. Further, the simulation may not accurately or realistically represent how a person would move or adjust the position or orientation of an object around the time of a handover action, particularly as the robot nears the human hand. For example, some people might back away from the gripper to reduce a risk of contact, while other people might attempt to push the object into the gripper in order to “help” the robot during the handover action. There may be other behaviors of a human, or other organism being simulated, that may not be accurately represented in prior simulations as well.

Approaches in accordance with various embodiments can provide for simulations in which a robot, or a control system or algorithm for a robot or automated assembly, can learn to exchange objects with humans seamlessly and safely for a variety of human-robot interactions (HRI). In at least one embodiment, this can include use of a framework for accurately simulating human-robot object handovers. This framework can also be used to provide a simulation-based benchmark for training and evaluating robot policies for handovers or other such interactions. Such an approach can provide for accurate simulation of a realistic human agent, as well as interactions of this agent with a robot during an interaction, such as may relate to a handover process.

An example simulation-based benchmark for human-to-robot (H2R) object handovers can be described with respect to a handover action as illustrated and described with respect to FIGS. 1A-1D, where the task of a robot, or automated receiver, is to take over an object handed by a human or other such giver. Such simulation can dedicate specific resources and functionality to accurately depicting motion and actions of a human hand, as well as the motion of an object resulting from a simulated human giver. In at least

one embodiment, such realism can be achieved, at least in part, through use of a motion capture system and process, as may capture images or other sensor data as illustrated in FIGS. 1A-1D and FIGS. 2A-2D. Such a system can be used to capture motion for interactions, such as human hand-object interactions including real human hand motion during handover attempts, with high fidelity. For example, a camera may capture video, or a sequence of images or other sensor data (e.g., RGB-D or LIDAR data) of a human hand performing various actions. This can include, for example, a human hand performing a handover as illustrated in FIGS. 1A-1D. This may also, or alternatively, include other human hand behavior, such as for a hand picking up an object for a handover as illustrated in FIGS. 2A-2D, which can provide for more accurate simulation of a hand for various objects during such an interaction. For example, instead of modeling the handover itself, as described with respect to FIGS. 1A-1D, a simulation process can focus on an earlier portion of that handover process, such as before the hand has contacted the object as illustrated in image 200 of FIG. 2A. This can capture motion of the hand moving towards the object 204 as illustrated in FIG. 2A, including how the hand may move to avoid other objects 206 in the environment. The process can capture the hand grasping the object as illustrated in image 220 of FIG. 2B, picking up the object as illustrated in image 240 of FIG. 2C, and moving the object toward an appropriate handover location or region as illustrated in image 260 of FIG. 2D. At least some of these captured images or clips showing typical human motion of a human giver that can be used in a physics simulator. This capability can then be used to create a physics-simulated benchmark environment for training and evaluating robot receivers with standardized (or other) evaluation protocols and metrics. In one example, an environment was used to obtain data for 1,000 handover scenes from 10 subjects handing over 20 different objects in the real world. A simulation benchmark "HandoverSim" can be used for human-to-robot object handovers that leverages such motion capture data for a hand grasping objects to synthesize realistic motion of human givers. Such an approach can be used with various robots or automated (or partially automated) devices, systems, or components, as may include domestic robots, assistive robots for older adults and people with disabilities, and collaborative robots in manufacturing.

When training or instructing a robot to perform an interaction such as an object handover, it can be beneficial to have the robot be able to exchange an object with a human in a way that can be both smooth and intuitive for the human, such that there is no unexpected movement or action that may negatively impact the interaction. Various models of humans, human hands, or other such giver elements may be used, but it can be challenging to cause these models to behave realistically in simulation, including moving not just in realistic motions with realistic poses, but with realistic physics. As mentioned, motion capture can be used to determine realistic behavior of a giver such as a human hand. In order to avoid issues discussed herein with respect to capturing handover actions, however, various approaches can use aspects of the interaction performed by the giver element, up to approximately the point of handover, or where handover could occur, and then transfer that learning to the simulation. As discussed herein, a significant portion of the training of the robot for a handover is to be able to understand and anticipate realistic human motion during a handover, and to be able to approach and grasp the object, then do another action after the human hand releases the object. The type of action to be performed afterward is not

of concern for the handover (at least in most situations) so the simulation can stop at the point where the robot has grasped and is supporting the object. Further, since the robot will generally pause in a grasp position for a period of time to allow the hand to release the object and back away, there is no real need (again, in at least most simulations) to model the hand moving away, as the robot will not move or adjust during this time unless triggered by another action such as a contact as discussed herein. Thus, modeling the motion of the human hand and object up to the point of grasp can be sufficient for handover simulation purposes. Accordingly, motion capture can be performed for a person grasping an object and putting in a position for a handover without any need for an actual robot to be present or an actual handover to be performed. Image or sensor data can be captured of one or more hands with respect to one or more objects, and this motion data can be transferred into a physics simulator using a framework such as that presented herein.

In at least one embodiment, the models of the object and the human hand (or hands for a large or heavy object) can be driven in a physics simulator so that they follow realistic trajectories as determined from the motion capture data, including realistic physical behavior in addition to kinematic and geometric behavior. In such a simulation, it can be important to make sure that the model of the hand does not penetrate the model of the object, as may happen due to noise in the motion capture data, imprecise modeling, or imprecise trajectories, which could otherwise result in at least some amount of instability in the physics simulation. Accordingly, approaches in accordance with various embodiments can disable collision detection between the human hand and the object, or can at least set minimum collision thresholds such that collisions between the hand and object will only trigger a failure if there is more than a threshold amount of contact (whether in surface contact, depth of contact, or length of contact, among other such options). Collision detection can still allow for the detection of collisions between the robot gripper and the object for use in triggering a handover action in the simulation. In one or more embodiments, separate controllers can be used to control the hand and object directly, so that they can separately (but in tandem) cause the object and hand to move as they would if the hand were actually carrying the object in a physical environment. When a collision between the object and the robot gripper is determined in the simulation, the controllers for the object can be disabled such that the object becomes (e.g., completely) passive, and can drop to the ground (or table, etc.) if not properly held by the robot gripper. It can be up to the robot control algorithm to ensure that the object is properly held, which can be separate from the simulation environment for the handover (including grasping an object from a human).

As mentioned, a significant issue to be addressed in providing a simulation-based benchmark for human-robot object handovers is how to simulate a realistic human agent and its interaction with a robot during, for example, a handover process. Photorealistic and physics-accurate simulation of humans has been widely studied in graphics, but is still under active research as accurate simulations are difficult to achieve. Furthermore, object handovers involve contact-rich interactions between human hands and objects. A high-fidelity simulation can require substantial physics modeling and sophisticated simulation capabilities on soft body dynamics.

A framework such as HandoverSim, discussed herein, can provide a simulation framework and benchmark for actions such as human-to-robot object handovers. Some of the

examples presented herein focus on a less-explored but challenging human-to-robot (H2R) paradigm, where a robot can be tasked with taking and holding an object handed over by a human. This can include, for example, focusing specifically on a realism of hand motion in simulating a human giver. In at least one embodiment, a motion capture dataset of one or more humans grasping objects and performing handover attempts can be leveraged to build a simulation environment where the motion of the human giver is driven by the captured motion.

An example handover simulation can assume that a given scene or environment includes a human giver and a robot arm receiver facing each other with a table in between, and a set of objects initially placed on the table, as illustrated in FIG. 1A. The human giver can pick up an object from the table with a single hand (right or left) and offer that object to the robot. A camera or sensor system in communication with the robot can capture sensor data to observe the actions of the human, as well as information such as a current state of the scene or environment, which may include positions or motions of other objects. This captured state data can be used by a control system to cause or enable the robot to react quickly, to one or more changes in state that may occur sequentially or concurrently, to eventually take over the object from the human hand of the giver.

To simulate the physical interaction of such a process, a simulation environment can be generated using an appropriate physics simulator, such as the PyBullet physics engine. An appropriate robot or automated assembly can be modeled or used for the simulation, such as a model of a Franka Emika Panda with a seven degrees of freedom (7-DoF) arm and a 2-DoF parallel-jaw gripper, although other types of end effectors can be used as well for other robot selections. It should be noted that a handover process is typically a two-agent game, involving a human and a robot, although there may be other or additional agents in other environments or simulations. In an example benchmark, a robot may be expected to be the only controllable agent, and expected to be able to move or operate freely within its own physical (or controlled) limits. Such an approach cannot readily be used to simulate a realistic human giver, particularly with respect to natural motion and interaction with a robot. Below we describe our approach.

To simulate realistic human motion with respect to an object and an offering, a recent human grasping dataset (e.g., DexYCB) can be used that illustrated captured data for real motion of human subjects picking up an object from a table of objects, as illustrated in FIGS. 2A-2D, and handing that object to an imagined partner across the table. In such a scenario, a handover does not actually need to occur, but this data can include motion up to a position where a handover can occur, such as is illustrated in image 260 of FIG. 2D. This data can thus represent natural human motion and actions for a handover, as well as motion and actions leading up to that handover. This may include a capture starting from a giver in a relaxed pose, and ending with a hand of the giver holding the object in the air, waiting for a receiver to acquire that object as illustrated in FIG. 2D. Each image or capture can provide framewise 3D pose data for both the hand in use, as well as one or more objects relative to the hand. This data can serve as a basis for modeling or simulating motion of a human giver in a “pre-handover” phase, or period of time before a gripper of the receiver contacts the object in the giver’s hand. Such captured data can be used to drive the human giver’s motion in simulation. This can include importing appropriate object and hand models into a simulation environment, as may include models for a number

(e.g., 20) of different types of objects. The pose of each object can be represented by a pose (e.g., a 6D pose) of a rigid mesh model, among other such options. A human hand pose can be represented by a deformable model, such as a MANO mesh model that is parameterized by two low-dimensional embeddings for shape and articulation respectively. A hand model can be loaded into PyBullet, for example, which can transform the hand model into an articulated rigid body after appropriate shape parameter deformation.

One approach to reproducing captured motion in simulation would be to learn a control policy to actuate the human hand to pick up the object with close fidelity to these various captured trajectories, although learning to control a dexterous hand to manipulate objects has been notoriously challenging and typically requires significant engineering effort with respect to the hand’s physics model. Approaches in accordance with at least one embodiment can, instead of relying on the human hand to physically move an object for a handover, augment the object models by adding additional actuators to their base, such that their (e.g., 6D) pose can be directly actuated by controllers in simulation. Such a simulation environment allows for direct control of the human hand model and the grasped object model to cause those models to move simultaneously according to their captured trajectories. Due at least in part to noise that may be present in motion capture data, the data may contain moderate interpenetration between the hand and object model during grasping. To account for this potential interpenetration, a simulation can disable any collision detection between the human hand and object being simulated, which can help to avoid artifacts caused by unstable simulation. For example, motion of the hand 202 and motion of the object 204 can be simulated separately between FIGS. 2C and 2D, where both can follow a respective trajectory, but the simulation allows for slight variances in position that might otherwise be prevented due to improper contact or interpenetration determined from the noisy data. For each capture, a handover trial may be simulated by “replaying” the motion from the first frame. After reaching the last frame of motion data, the hand model and object model can be forced to stay in this “final” end pose, such as is illustrated in FIG. 1A, where the giver is simulated to be waiting for the receiver to grasp the object.

Once a simulation framework is able to simulate a human presenting an object for handover, for example, a remaining task can be to simulate the human releasing the object to the robot, corresponding to a “physical handover” phase. In a simulation where the pose of an object has been actively controlled to this point, the controller can be disabled to simulate a release. A condition that can be used to triggering a release may correspond to a contact force being detected between the gripper “fingers,” or other articulable portions of an end effector, and a presented object. This may correspond to the human receiving force feedback from the object that is induced by the robot contacting the object. During a simulation, “compulsive” releases can also be modeled, such as where the object drops from the human hand due to a strike by the robot arm. In such cases, a release can be triggered whenever a contact force is detected between any part of the robot body and the presented object, at least where that contact lasts for at least a minimum period or length of time, such as for at least 0.2 seconds. Once triggered, the object can become completely passive and can be subject to forces from the robot and gravity. Such a simulation may simulate human motion that is non-adaptive to the motion of the robot, which may seem to be naive but may be reasonable in at least some scenarios where the

human is cognitively engaged in one or more other tasks and has to exchange objects with robots “blindly.” Such a simulation environment can also provide a useful platform for learning adaptive human agents. In at least some embodiments, realistic human hand motion during object release and post-release may not be simulated due to the lack of such data in the dataset.

In an example implementation of a handover simulation framework, a benchmark environment can be used to evaluate one or more robot receiver policies. A task to be evaluated can be formalized as one or more standard reinforcement learning (RL) problems, and the benchmark environment can be implemented using, for example, using an OpenAI Gym API. At each time step, the robot agent can observe the current state of the environment $s_t \in S$, and generates an action $a_t \in A$ from its policy. The robot then executes at in the environment, which returns a new state s_{t+1} and optionally a scalar reward r if an RL algorithm is used. A in this example represents a 9-dimensional continuous space where each action specifies a target joint position for the position control of the robot arm (7 DoF) and gripper (2 DoF). S may vary depending on one or more benchmark settings or scenarios, as may relate to an object pose or point cloud. For quantitative evaluation, when a task has completed successfully or failed can be programmatically defined. This can include claiming that a success is achieved when all of a set of three conditions are met, as may include: one or more gripper fingers being in contact with the handed-over object, the position of the gripper link lies with a goal or target region for the interaction, and this contact and positioning holding true continuously for at least 0.1 seconds.

In at least one embodiment a spherical (or other similarly shaped 3D) goal region can be established in close proximity to a front of the robot arm, which can help to prevent a success case being determined where the robot has reached the object but becomes stuck due to an unnatural pose configuration. The robot instead should be able to pull back the object to close proximity after taking hold of the object, and potentially be able to use the object to perform other tasks. On the other hand, a failure case can be detected when one of a set of conditions is met, as may include: any part of the robot body contacting any part of the human hand (or other portion of a human); at least one of the gripper fingers not being in contact with the handed-over object and the object being in contact with the table or support, being in contact with one or more other objects, having its center fall below the height of the table surface; or a maximum time being reached. This first condition (referred to as “contact”) prohibits robot-human contact, which can help to avoid potential harm to a human, such as may result from a human hand being pinched by the gripper, and ensure a safe handover process. The second condition (referred to as “drop”) can prevent the robot from dropping the object. The third condition (referred to as “timeout”) can help to ensure that the task is completed within a reasonable length of time.

For evaluation of such an approach, object handover can be regarded as a multi-objective task in HRI, and for an example benchmark metrics can be reported on efficacy, efficiency, and safety. Using definitions of success and failure such as those presented above, an episode can be terminated whenever a success or failure is detected. Taking such an approach will ensure that a given episode will only belong to a success case or a failure case, but not both, such as where the robot will not be allowed to complete a given task after touching a human hand. To evaluate efficacy, the success rate can be calculated in one or more embodiments

over all test episodes, with the failure rate also being calculated for specific failure causes, such as “contact,” “drop,” and “timeout” failure cases. For efficiency, the mean completion time can be calculated in one or more embodiments over the successful episodes, while not including episodes of failure in this metric. During evaluation, the completion time can further be divided into execution time and planning time. The execution time can refer to the accumulated time during which the robot is physically moving, and may depend only, or at least primarily, on the number of time steps of the episode. The planning time (“plan”) can include the accumulated wall time on running the policy function. A failure rate due to robot-human contact can be evaluated as a safety metric.

FIG. 3A illustrates an example system 300 that can be used to model realistic motion of a human hand for a human-robot interaction that can be used in accordance with various embodiments. It should be understood that such interactions can be performed with both hands or other portions of a human body, and are also not limited to (inter) actions such as object handovers which are presented as a primary example herein. In this example system 300, one or more cameras 302 or sensors can be positioned with respect to a physical environment 304 that includes, or will include, a human hand 306 and at least one physical object 308 with which that hand is to interact, such as to grasp and move the object. The camera(s) 302 can capture image data 310 (e.g., a sequence of images or video) representative of the state of the physical environment, including position, orientation, and pose information for the hand and object in the physical environment. A camera 302 used for such purposes can include a camera for capturing two-dimensional images, a camera for capturing stereoscopic images, or a camera for capturing images that include color and depth information (e.g., RGB-D images). Other sensors or devices may be used to capture a representation of the environment as well, as may include depth sensors, ultrasonic sensors, LIDAR scanners, and the like.

The captured image or sensor data can be provided as input to a motion modeling system 320. This input can be updated and provided as appropriate, such as for every captured frame of a video stream or in response to any detected motion in the environment. In some instances, at least some of the image or sensor data provided to a motion modeling system 320 may be provided by, or obtained from, a separate party, entity, or source that captured or generated the image data. In this example, the motion modeling system 320 can pass at least some of the image data 310 to an image analyzer that can recognize or identify the hand and the object of interest in individual images or video frames where at least a sufficient portion of the hand and object are represented. This can include, for example, a computer vision algorithm or neural network trained to recognize the hand and object in an image, and determine a location or segmentation of the image corresponding to the hand and object. This information can be passed to a motion modeling component 324, which can use position, orientation, pose, or other such information across a sequence of images or video frames to model motion of the hand and object in the sequence. The motion modeling component can work with, or include, a physics engine 334 that can provide for realistic modeling of the motion based upon physical forces and other such factors. As discussed elsewhere herein, representations of motions of the hand and the object may be performed separately in at least some embodiments. Motion models for the hand and the object can be updated for individual motions determined for individual sequences to

11

generate at least one motion model for a human hand, and may also generate at least one model for an object held in a human hand, to provide for predictions of realistic motion and behavior of a hand and object during an interaction, such as an object handover.

Once these motion models have been generated, they can be provided to a simulator **330** or stored to a motion model repository **340** accessible to a simulator. A simulation manager **332** of the simulator can be responsible for coordinating simulations executed by a simulation application **334**, with results evaluated by an evaluator. In at least some embodiments, a simulator **334** can select appropriate models **342** (e.g., deformable meshes) for a hand, object, and robot to be simulated in a virtual environment. The simulator **334** can also determine, from a respective table in a constraint database **344** or other such location, one or more constraints to be enforced during the simulation. For a given type of simulation, the simulator can determine or receive indication of an interaction to be simulated, such as an object handover from a human to a robot, and can position the hand, object, and robot accordingly. The simulator can use the motion models for the hand and object, generated by the motion modeling system **320**, to simulate motion or behavior of the hand and the object during a handover operation. As discussed, the hand and object may be controlled separately according to respective motion models, with collision detection disabled therebetween. The simulator **334** can cause the virtual robot to select a grasp position and orientation, and determine an optimal (or at least highest ranked or scored) path to that grasp position. The simulator **334** can operate the simulation until an appropriate end criterion is satisfied, such as for a target object-robot contact being achieved for a successful handover, or an undesired human contact or dropping of the virtual object in the simulation for a failed event. In this example, upon reaching an end criterion, information for the simulation can be passed to an evaluator **336**, which can determine various metrics, as discussed elsewhere herein, for a successful or unsuccessful action or interaction. These metrics can be used to update parameters for a robot control model, or other such control algorithm or process, to attempt to minimize the likelihood of an unsuccessful handover or interaction. Once the control algorithm is determined, through simulations, to have at most an acceptable likelihood of an unsuccessful interaction, that control algorithm can be provided for use by a robot in performing actual handovers, or other human-robot interactions, in a real-world, physical environment.

FIG. 3B illustrates components of an example system **350** that can be used to perform actions such as automated handovers, in accordance with at least one embodiment. Such a system **350** can take advantage of human and object motion or behavior models, such as those generated using a process **300** described with respect to FIG. 3A. In this example, one or more cameras **390** can be used to capture image data **362** (e.g., a sequence or images or video) representative of a physical environment including a robot **354** and at least one object **352** with which that robot is to interact. The camera can include a camera or sensor such as those discussed above with respect to FIG. 3A. This captured environmental data can be fed as input to a state monitor **374** of a robot control system **370**. This input can be updated and provided as appropriate, such as for every captured frame of a video stream or in response to any detected motion in the environment.

In this example system **350**, the state monitor **374** can use this (and any other relevant) input to generate a representation of the current state of the environment. This can

12

include, for example, a current position and orientation of a robot **354** or portion of a robot, such as an end effector **356** or gripper, as well as that of at least one object **352** with which the robot **354** is to interact. The state monitor **374** can generate any appropriate type of representation, such as a 3D model or point cloud representing the current state of the environment. This determined or predicted state information can be provided to various components of the control system **370**, such as a grasp predictor **376** and a motion predictor **378**. As discussed herein, at each time step in this process, or at least at periodic time steps, a grasp predictor **376** for a handover operation (or other location and orientation predictor for other types of tasks) can determine one or more target grasps, or other target locations and orientations of at least an end effector **356** (or other portion) of the robot **354** to at least start a determined task, action, or other interaction with the object **352**. This can include updating a determined grasp in response to any change in the environment, as may correspond to a change in position or orientation of the robot **354**, the object **352**, a hand or other support holding the object, or another object or entity in the environment, among other such options. Updating the target grasp can help to ensure that the end effector of the robot does not perform undesired actions, such as to contact a human hand or obstruct a view of the camera **390**, as well as to ensure that the grasp of the object **352** is sufficient to hold that object **352** without risk of dropping or damaging the object, for any potential changes in position or orientation of that object **352**.

At each time step, a motion predictor **378** can also analyze the current state data, as well as any updates to the target grasp determination. As mentioned, in some embodiments the grasp predictor **376** can provide a full set of potential grasps, and the motion predictor **378**, which can include a motion path predictor or MPC, can select the current target grasp to be used. In some embodiments, all potential grasps in a set can be treated equally, as long as those grasps satisfy one or more minimum criteria, while in other embodiments there may be some weighting or ranking of the relevant grasps, such as may be determined based upon factors such as distance or strength of grasp position. The motion predictor **378** can consider any constraints on the motion, and can use a motion planner or optimizer application, algorithm, or neural network, for example, to predict an optimal path between a current position and orientation of the robot and a current position and orientation of the object, as well as the hand holding the object. This prediction can also take into account predicted behavior of the human hand, as well as the impact of that behavior on the object, such as by using at least a hand motion or behavior model generated using a process such as that described with respect to FIG. 3A. As discussed herein the optimization can take other factors or constraints into account as well, such as to prefer smooth or straight-line motion, or to avoid camera obstructions. In at least some embodiments, state data may be received from the robot at a higher frequency than state data is updated based upon camera data, such that motion data may be updated more frequently than grasp data in some embodiments. The motion predictor **378** can then determine a sequence of motions, or a path of motion to be performed over a number of future time steps. The motion predictor **378** can then provide information for this sequence or future motions, a first subset of these future motions, or only a first predicted motion to a robot controller **372** of the control system **370**. In this way, motion will be optimized by looking over a range of future motions, rather than just a current motion, which can help with factors such as smooth-

ness of motion and predictability. The robot controller 372 can then determine instructions to cause the robot to perform an action corresponding to that motion, if not already provided by the motion predictor 378, and can send those instructions to the robot 354, where a drive system 360 or other mechanical control of the robot can cause the robot 354 to perform that motion, such as to perform a determined motion for a current time step to bring an end effector 356 of the robot to a target position relative to the object. In many instances this will result in the end effector being moved toward the target grasp position and orientation, but due to factors such as motion of the hand or object may involve moving away from the object to avoid a collision or obstruction relative to the hand or object. In this example, the end effector 356 may have one or more contact sensors that can provide contact data to a contact detection system 358, which can provide this information to the drive system 360 and/or robot controller 372 of the control system. In some embodiments, the drive system 360 may have a built in safety mechanism that causes the robot to stop if contact is detected. In some embodiments, a robot controller 372 may alternatively, or additionally, update state information to include the contact information, which can then be provided to the motion predictor to determine any subsequent actions or motions to be taken, such as to move away if the contact is not a desired contact. In some embodiments, if the end effector is in the target grasp position and orientation and an anticipated contact is detected, then the motion predictor 378 may provide information to the robot controller 372 to cause the robot to stay in that position for at least a determined period of time to allow for the hand to move away and for the handover action to be completed. In other embodiments, a robot controller 372 may make such a determination. Such a system can be used to control a robot to perform other tasks as well, such as may require a determined position or orientation to be determined with similar, or alternative, constraints being applied. In some embodiments, a grasp predictor 376, motion predictor 378, or robot controller 372 may need to determine applicable constraints for a given task or action, as may be obtained from a constraint repository 380 or other such location.

FIG. 4 illustrates an example process 400 for providing a simulation environment for human-robot interactions that can be used in accordance with various embodiments. It should be understood that for this and other processes discussed herein, there can be additional, fewer, or alternative steps performed in similar or alternative orders, or at least partially in parallel, within the scope of the various embodiments unless otherwise specifically stated. Further, although this process is described with respect to robots and handover actions, aspects of such a process can be used with other types of automation and for other types of tasks as well within the scope of the various embodiments. In this example, one or more cameras (or sensors) are positioned 402 with respect to a physical environment that does, or will, include a human hand and an object with which the human hand is to interact. Image (or sensor) data can then be captured 404 that is representative of various instances of the hand, or multiple hands, grasping the object, or different objects, and moving the object to one or more positions and orientations at which an action, such as a handover action, can be performed. A determined number of instances, such as around 1,000, may be performed and captured, as may include at least a minimum number of different types of objects and different handover or target locations, poses, and orientations. At least a subset of this captured image data can be analyzed 406 to identify at least a position and pose of the

hand and the object in individual images or video frames. This can include identifying a location or coordinates, such as a bounding box, or determining a segmentation of these images that identify segments of the image corresponding to the hand and object. In other embodiments, image features may be extracted and identified for the hand and object and then encoded into a latent space for modeling and motion/behavior prediction, among other such options. A physics engine can then be used 408 with a modeling process to model realistic motion or behavior of the hand and the object from at least a grasp position to a handover position, or at least a portion of motion prior to arriving at a handover position. One or more motion models generated for this motion or behavior can then be provided 410 to a simulator for human-robot interactions, or otherwise stored for such potential usage.

A simulator for a given task, such as a human-robot handover of an object, can generate 412 a simulation environment that includes a virtual hand, model, and object. In at least one embodiment, this simulation can be performed to optimize a control algorithm for a specific type of robot or automated assembly, so the simulator can select a model for a virtual robot that corresponds to the specific type of robot to be controlled. The simulator may also choose appropriate models for a hand and an object from a set of models, in order to provide for a variety of different potential hand and object shapes for which the robot control algorithm should account. Multiple simulations can be performed, which may include a number of different hand and object models being selected. In a given simulation, the virtual robot can be caused 414 to move to a position and orientation from which the robot can grasp the object without contacting the human hand or violating another constraint, such as to obstruct the camera. The motion determined for the robot can be based at least on predicted human motion or behavior and its impact on the object, as may be determined using the generated motion or behavior models generated from the captured motion data. At each time step in this example, a determination can be made 416 as to whether a target contact between the robot and the object has occurred or been detected, such as where an end effector of a robot has made an acceptable type of contact with the object to successfully complete the handover action. As discussed, this may include the target contact occurring for at least a minimum period or length of time, such as may ensure that the robot has sufficient time and ability to grasp the object to avoid the object falling if the hand releases the object. If not, the process can continue and the virtual robot can be controlled to attempt to arrive at such a location. If such contact is detected, then a robot control model (or algorithm or process, etc.) can be updated 418 based, at least in part, upon this successful handover action, such as to adjust parameters of a neural network used to predict or infer optimal robot motion. Although not illustrated in this example flowchart, there may be other criteria that cause a given simulation instance to be ended, such as where a constraint is not complied with, or an undesired contact occurs as discussed herein. Information for such a failure may also be used to update the control algorithm, such as to adjust parameters to try to minimize a value of a loss function being used to train a motion control model. Instances of this simulation can be performed until, for example, it is determined that the robot control algorithm has at most a probability of failure that is below an acceptable failure threshold, or has at least a probability of success that is at or above a target success threshold (or other such metric).

15

FIG. 5 illustrates an example process 500 for controlling a robot to perform a handover action based, at least in part, upon predicted motion or behavior of a human hand as modeled and accounted for in training of a control algorithm, such as is described in the process 400 of FIG. 4. In this example process, a handover action to be performed by a robot, or other automated or semi-automated assembly, can be determined 502, where this handover action can involve the robot taking an object currently, or to be, held by a human hand. A current state of the physical environment, including the robot, hand, and object can be determined based at least on image or sensor data captured of the physical environment, as well as orientation or motion data provided by the robot. Based at least in part upon this current state, as well as modeled human motion or behavior determined from simulations and accounted for in a robot control model, as discussed with respect to FIG. 4, an optimal location and orientation for the robot to perform the handover action can be determined 504. For this current state, and based at least on this predicted human behavior, and optimal sequence of motions can be predicted 506 to direct the robot to a currently-determined optimal location and orientation for the handover. At least a first motion, or subset of motions, of this sequence can be performed 508, and a determination made 510 as to whether an end effector, or other portion of this robot, has arrived at this target location and orientation to perform the handover. If not, this process can continue (unless another end criterion is satisfied as discussed herein, such as human-robot contact or the object being dropped or damaged). If the end effector of the robot is determined to have successfully arrived at the target location and orientation then the robot can be caused 512 to perform the handover action at that location and orientation, such as may involve holding the object for a minimum period of time to allow the human hand to move away, or detecting such movement of the human hand, and then performing a subsequent task based on successful completion of the handover.

As discussed, various approaches presented herein can be lightweight enough to execute on a device such as a client device, such as a personal computer or gaming console, in real time. Such processing can be performed on content that is generated on, or received by, that client device or received from an external source, such as streaming sensor data or other content received over at least one network. In some instances, the processing and/or determination of this content may be performed by one of these other devices, systems, or entities, then provided to the client device (or another such recipient) for presentation or another such use.

As an example, FIG. 6 illustrates an example network configuration 600 that can be used to provide, generate, modify, encode, and/or transmit data or other such content. In at least one embodiment, a client device 602 can generate or receive data for a session using components of a control application 604 on client device 602 and data stored locally on that client device. In at least one embodiment, a control application 624 (e.g., a robot control application) executing on control server 620 (e.g., a cloud server or edge server) may initiate a session associated with at least client device 602, as may use a session manager and user data stored in a user database 634, and can cause content 632 to be determined by a content manager 626. A control application 630 may obtain image data for a scene or environment, and work with an instruction module 626, simulation module 628, control module 630, or other such component to simulate behavior of a human in an interaction, update a control model based at least partially on that simulation, then

16

generate a sequence of motions or accelerations to be performed by a robot or automated system 670 in that environment corresponding to that instruction. In this example, the content may relate to conditions about the environment, a sequence of discrete motions, machine-executable instructions, or other such data discussed or suggested herein. At least a portion of that content may be transmitted to client device 602 using an appropriate transmission manager 622 to send by download, streaming, or another such transmission channel. An encoder may be used to encode and/or compress at least some of this data before transmitting to the client device 602. In at least one embodiment, content may alternatively be provided to an automated system 670 for performance, with monitoring or instruction data coming from client device 602 and monitoring or status data being provided to client device 602, whether directly or across at least one network 640. In at least one embodiment, client device 602 receiving such content can provide this content to a corresponding control application 604, which may also or alternatively include a graphical user interface 610, planning module 612, and control module 614 or process. A decoder may also be used to decode data received over the network(s) 640 for presentation via client device 602, such as image or video content through a display 606 and audio, such as sounds and music, through at least one audio playback device 608, such as speakers or headphones. In at least one embodiment, at least some of this content may already be stored on, rendered on, or accessible to client device 602 such that transmission over network 640 is not required for at least that portion of content, such as where that content may have been previously downloaded or stored locally on a hard drive or optical disk. In at least one embodiment, a transmission mechanism such as data streaming can be used to transfer this content from server 620, or user database 634, to client device 602. In at least one embodiment, at least a portion of this content can be obtained or streamed from another source, such as a third party service 660 or other client device 650, that may also include a content application 662 for generating or providing content. In at least one embodiment, portions of this functionality can be performed using multiple computing devices, or multiple processors within one or more computing devices, such as may include a combination of CPUs and GPUs.

In this example, these client devices can include any appropriate computing devices, as may include a desktop computer, notebook computer, set-top box, streaming device, gaming console, smartphone, tablet computer, VR headset, AR goggles, wearable computer, or a smart television. Each client device can submit a request across at least one wired or wireless network, as may include the Internet, an Ethernet, a local area network (LAN), or a cellular network, among other such options. In this example, these requests can be submitted to an address associated with a cloud provider, who may operate or control one or more electronic resources in a cloud provider environment, such as may include a data center or server farm. In at least one embodiment, the request may be received or processed by at least one edge server, that sits on a network edge and is outside at least one security layer associated with the cloud provider environment. In this way, latency can be reduced by enabling the client devices to interact with servers that are in closer proximity, while also improving security of resources in the cloud provider environment.

In at least one embodiment, such a system can be used for performing graphical rendering operations. In other embodiments, such a system can be used for other purposes, such

as for providing image or video content to test or validate autonomous machine applications, or for performing deep learning operations. In at least one embodiment, such a system can be implemented using an edge device, or may incorporate one or more Virtual Machines (VMs). In at least one embodiment, such a system can be implemented at least partially in a data center or at least partially using cloud computing resources.

Inference and Training Logic

FIG. 7A illustrates inference and/or training logic 715 used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic 715 are provided below in conjunction with FIGS. 7A and/or 7B.

In at least one embodiment, inference and/or training logic 715 may include, without limitation, code and/or data storage 701 to store forward and/or output weight and/or input/output data, and/or other parameters to configure neurons or layers of a neural network trained and/or used for inferencing in aspects of one or more embodiments. In at least one embodiment, training logic 715 may include, or be coupled to code and/or data storage 701 to store graph code or other software to control timing and/or order, in which weight and/or other parameter information is to be loaded to configure, logic, including integer and/or floating point units (collectively, arithmetic logic units (ALUs)). In at least one embodiment, code, such as graph code, loads weight or other parameter information into processor ALUs based on an architecture of a neural network to which the code corresponds. In at least one embodiment, code and/or data storage 701 stores weight parameters and/or input/output data of each layer of a neural network trained or used in conjunction with one or more embodiments during forward propagation of input/output data and/or weight parameters during training and/or inferencing using aspects of one or more embodiments. In at least one embodiment, any portion of code and/or data storage 701 may be included with other on-chip or off-chip data storage, including a processor's L1, L2, or L3 cache or system memory.

In at least one embodiment, any portion of code and/or data storage 701 may be internal or external to one or more processors or other hardware logic devices or circuits. In at least one embodiment, code and/or code and/or data storage 701 may be cache memory, dynamic randomly addressable memory ("DRAM"), static randomly addressable memory ("SRAM"), non-volatile memory (e.g., Flash memory), or other storage. In at least one embodiment, choice of whether code and/or code and/or data storage 701 is internal or external to a processor, for example, or comprised of DRAM, SRAM, Flash or some other storage type may depend on available storage on-chip versus off-chip, latency requirements of training and/or inferencing functions being performed, batch size of data used in inferencing and/or training of a neural network, or some combination of these factors.

In at least one embodiment, inference and/or training logic 715 may include, without limitation, a code and/or data storage 705 to store backward and/or output weight and/or input/output data corresponding to neurons or layers of a neural network trained and/or used for inferencing in aspects of one or more embodiments. In at least one embodiment, code and/or data storage 705 stores weight parameters and/or input/output data of each layer of a neural network trained or used in conjunction with one or more embodiments during backward propagation of input/output data

and/or weight parameters during training and/or inferencing using aspects of one or more embodiments. In at least one embodiment, training logic 715 may include, or be coupled to code and/or data storage 705 to store graph code or other software to control timing and/or order, in which weight and/or other parameter information is to be loaded to configure, logic, including integer and/or floating point units (collectively, arithmetic logic units (ALUs)). In at least one embodiment, code, such as graph code, loads weight or other parameter information into processor ALUs based on an architecture of a neural network to which the code corresponds. In at least one embodiment, any portion of code and/or data storage 705 may be included with other on-chip or off-chip data storage, including a processor's L1, L2, or L3 cache or system memory. In at least one embodiment, any portion of code and/or data storage 705 may be internal or external to on one or more processors or other hardware logic devices or circuits. In at least one embodiment, code and/or data storage 705 may be cache memory, DRAM, SRAM, non-volatile memory (e.g., Flash memory), or other storage. In at least one embodiment, choice of whether code and/or data storage 705 is internal or external to a processor, for example, or comprised of DRAM, SRAM, Flash or some other storage type may depend on available storage on-chip versus off-chip, latency requirements of training and/or inferencing functions being performed, batch size of data used in inferencing and/or training of a neural network, or some combination of these factors.

In at least one embodiment, code and/or data storage 701 and code and/or data storage 705 may be separate storage structures. In at least one embodiment, code and/or data storage 701 and code and/or data storage 705 may be same storage structure. In at least one embodiment, code and/or data storage 701 and code and/or data storage 705 may be partially same storage structure and partially separate storage structures. In at least one embodiment, any portion of code and/or data storage 701 and code and/or data storage 705 may be included with other on-chip or off-chip data storage, including a processor's L1, L2, or L3 cache or system memory.

In at least one embodiment, inference and/or training logic 715 may include, without limitation, one or more arithmetic logic unit(s) ("ALU(s)") 710, including integer and/or floating point units, to perform logical and/or mathematical operations based, at least in part on, or indicated by, training and/or inference code (e.g., graph code), a result of which may produce activations (e.g., output values from layers or neurons within a neural network) stored in an activation storage 720 that are functions of input/output and/or weight parameter data stored in code and/or data storage 701 and/or code and/or data storage 705. In at least one embodiment, activations stored in activation storage 720 are generated according to linear algebraic and or matrix-based mathematics performed by ALU(s) 710 in response to performing instructions or other code, wherein weight values stored in code and/or data storage 705 and/or code and/or data storage 701 are used as operands along with other values, such as bias values, gradient information, momentum values, or other parameters or hyperparameters, any or all of which may be stored in code and/or data storage 705 or code and/or data storage 701 or another storage on or off-chip.

In at least one embodiment, ALU(s) 710 are included within one or more processors or other hardware logic devices or circuits, whereas in another embodiment, ALU(s) 710 may be external to a processor or other hardware logic device or circuit that uses them (e.g., a co-processor). In at

least one embodiment, ALUs **710** may be included within a processor's execution units or otherwise within a bank of ALUs accessible by a processor's execution units either within same processor or distributed between different processors of different types (e.g., central processing units, graphics processing units, fixed function units, etc.). In at least one embodiment, code and/or data storage **701**, code and/or data storage **705**, and activation storage **720** may be on same processor or other hardware logic device or circuit, whereas in another embodiment, they may be in different processors or other hardware logic devices or circuits, or some combination of same and different processors or other hardware logic devices or circuits. In at least one embodiment, any portion of activation storage **720** may be included with other on-chip or off-chip data storage, including a processor's L1, L2, or L3 cache or system memory. Furthermore, inferencing and/or training code may be stored with other code accessible to a processor or other hardware logic or circuit and fetched and/or processed using a processor's fetch, decode, scheduling, execution, retirement and/or other logical circuits.

In at least one embodiment, activation storage **720** may be cache memory, DRAM, SRAM, non-volatile memory (e.g., Flash memory), or other storage. In at least one embodiment, activation storage **720** may be completely or partially within or external to one or more processors or other logical circuits. In at least one embodiment, choice of whether activation storage **720** is internal or external to a processor, for example, or comprised of DRAM, SRAM, Flash or some other storage type may depend on available storage on-chip versus off-chip, latency requirements of training and/or inferencing functions being performed, batch size of data used in inferencing and/or training of a neural network, or some combination of these factors. In at least one embodiment, inference and/or training logic **715** illustrated in FIG. **7a** may be used in conjunction with an application-specific integrated circuit ("ASIC"), such as Tensorflow® Processing Unit from Google, an inference processing unit (IPU) from Graphcore™, or a Nervana® (e.g., "Lake Crest") processor from Intel Corp. In at least one embodiment, inference and/or training logic **715** illustrated in FIG. **7a** may be used in conjunction with central processing unit ("CPU") hardware, graphics processing unit ("GPU") hardware or other hardware, such as field programmable gate arrays ("FPGAs").

FIG. **7b** illustrates inference and/or training logic **715**, according to at least one or more embodiments. In at least one embodiment, inference and/or training logic **715** may include, without limitation, hardware logic in which computational resources are dedicated or otherwise exclusively used in conjunction with weight values or other information corresponding to one or more layers of neurons within a neural network. In at least one embodiment, inference and/or training logic **715** illustrated in FIG. **7b** may be used in conjunction with an application-specific integrated circuit (ASIC), such as Tensorflow® Processing Unit from Google, an inference processing unit (IPU) from Graphcore™, or a Nervana® (e.g., "Lake Crest") processor from Intel Corp. In at least one embodiment, inference and/or training logic **715** illustrated in FIG. **7b** may be used in conjunction with central processing unit (CPU) hardware, graphics processing unit (GPU) hardware or other hardware, such as field programmable gate arrays (FPGAs). In at least one embodiment, inference and/or training logic **715** includes, without limitation, code and/or data storage **701** and code and/or data storage **705**, which may be used to store code (e.g., graph code), weight values and/or other information, including

bias values, gradient information, momentum values, and/or other parameter or hyperparameter information. In at least one embodiment illustrated in FIG. **7b**, each of code and/or data storage **701** and code and/or data storage **705** is associated with a dedicated computational resource, such as computational hardware **702** and computational hardware **706**, respectively. In at least one embodiment, each of computational hardware **702** and computational hardware **706** comprises one or more ALUs that perform mathematical functions, such as linear algebraic functions, only on information stored in code and/or data storage **701** and code and/or data storage **705**, respectively, result of which is stored in activation storage **720**.

In at least one embodiment, each of code and/or data storage **701** and **705** and corresponding computational hardware **702** and **706**, respectively, correspond to different layers of a neural network, such that resulting activation from one "storage/computational pair **701/702**" of code and/or data storage **701** and computational hardware **702** is provided as an input to "storage/computational pair **705/706**" of code and/or data storage **705** and computational hardware **706**, in order to mirror conceptual organization of a neural network. In at least one embodiment, each of storage/computational pairs **701/702** and **705/706** may correspond to more than one neural network layer. In at least one embodiment, additional storage/computation pairs (not shown) subsequent to or in parallel with storage computation pairs **701/702** and **705/706** may be included in inference and/or training logic **715**.

Data Center

FIG. **8** illustrates an example data center **800**, in which at least one embodiment may be used. In at least one embodiment, data center **800** includes a data center infrastructure layer **810**, a framework layer **820**, a software layer **830**, and an application layer **840**.

In at least one embodiment, as shown in FIG. **8**, data center infrastructure layer **810** may include a resource orchestrator **812**, grouped computing resources **814**, and node computing resources ("node C.R.s") **816(1)-816(N)**, where "N" represents any whole, positive integer. In at least one embodiment, node C.R.s **816(1)-816(N)** may include, but are not limited to, any number of central processing units ("CPUs") or other processors (including accelerators, field programmable gate arrays (FPGAs), graphics processors, etc.), memory devices (e.g., dynamic read-only memory), storage devices (e.g., solid state or disk drives), network input/output ("NW I/O") devices, network switches, virtual machines ("VMs"), power modules, and cooling modules, etc. In at least one embodiment, one or more node C.R.s from among node C.R.s **816(1)-816(N)** may be a server having one or more of above-mentioned computing resources.

In at least one embodiment, grouped computing resources **814** may include separate groupings of node C.R.s housed within one or more racks (not shown), or many racks housed in data centers at various geographical locations (also not shown). Separate groupings of node C.R.s within grouped computing resources **814** may include grouped compute, network, memory or storage resources that may be configured or allocated to support one or more workloads. In at least one embodiment, several node C.R.s including CPUs or processors may grouped within one or more racks to provide compute resources to support one or more workloads. In at least one embodiment, one or more racks may

also include any number of power modules, cooling modules, and network switches, in any combination.

In at least one embodiment, resource orchestrator **812** may configure or otherwise control one or more node C.R.s **816(1)-816(N)** and/or grouped computing resources **814**. In at least one embodiment, resource orchestrator **812** may include a software design infrastructure (“SDI”) management entity for data center **800**. In at least one embodiment, resource orchestrator may include hardware, software or some combination thereof.

In at least one embodiment, as shown in FIG. **8**, framework layer **820** includes a job scheduler **822**, a configuration manager **824**, a resource manager **826** and a distributed file system **828**. In at least one embodiment, framework layer **820** may include a framework to support software **832** of software layer **830** and/or one or more application(s) **842** of application layer **840**. In at least one embodiment, software **832** or application(s) **842** may respectively include web-based service software or applications, such as those provided by Amazon Web Services, Google Cloud and Microsoft Azure. In at least one embodiment, framework layer **820** may be, but is not limited to, a type of free and open-source software web application framework such as Apache Spark™ (hereinafter “Spark”) that may use distributed file system **828** for large-scale data processing (e.g., “big data”). In at least one embodiment, job scheduler **822** may include a Spark driver to facilitate scheduling of workloads supported by various layers of data center **800**. In at least one embodiment, configuration manager **824** may be capable of configuring different layers such as software layer **830** and framework layer **820** including Spark and distributed file system **828** for supporting large-scale data processing. In at least one embodiment, resource manager **826** may be capable of managing clustered or grouped computing resources mapped to or allocated for support of distributed file system **828** and job scheduler **822**. In at least one embodiment, clustered or grouped computing resources may include grouped computing resource **814** at data center infrastructure layer **810**. In at least one embodiment, resource manager **826** may coordinate with resource orchestrator **812** to manage these mapped or allocated computing resources.

In at least one embodiment, software **832** included in software layer **830** may include software used by at least portions of node C.R.s **816(1)-816(N)**, grouped computing resources **814**, and/or distributed file system **828** of framework layer **820**. The one or more types of software may include, but are not limited to, Internet web page search software, e-mail virus scan software, database software, and streaming video content software.

In at least one embodiment, application(s) **842** included in application layer **840** may include one or more types of applications used by at least portions of node C.R.s **816(1)-816(N)**, grouped computing resources **814**, and/or distributed file system **828** of framework layer **820**. One or more types of applications may include, but are not limited to, any number of a genomics application, a cognitive compute, and a machine learning application, including training or inferencing software, machine learning framework software (e.g., PyTorch, TensorFlow, Caffe, etc.) or other machine learning applications used in conjunction with one or more embodiments.

In at least one embodiment, any of configuration manager **824**, resource manager **826**, and resource orchestrator **812** may implement any number and type of self-modifying actions based on any amount and type of data acquired in any technically feasible fashion. In at least one embodiment,

self-modifying actions may relieve a data center operator of data center **800** from making possibly bad configuration decisions and possibly avoiding underused and/or poor performing portions of a data center.

In at least one embodiment, data center **800** may include tools, services, software or other resources to train one or more machine learning models or predict or infer information using one or more machine learning models according to one or more embodiments described herein. For example, in at least one embodiment, a machine learning model may be trained by calculating weight parameters according to a neural network architecture using software and computing resources described above with respect to data center **800**. In at least one embodiment, trained machine learning models corresponding to one or more neural networks may be used to infer or predict information using resources described above with respect to data center **800** by using weight parameters calculated through one or more training techniques described herein.

In at least one embodiment, data center may use CPUs, application-specific integrated circuits (ASICs), GPUs, FPGAs, or other hardware to perform training and/or inferencing using above-described resources. Moreover, one or more software and/or hardware resources described above may be configured as a service to allow users to train or performing inferencing of information, such as image recognition, speech recognition, or other artificial intelligence services.

Inference and/or training logic **715** are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic **715** are provided below in conjunction with FIGS. **7A** and/or **7B**. In at least one embodiment, inference and/or training logic **715** may be used in system FIG. **8** for inferencing or predicting operations based, at least in part, on weight parameters calculated using neural network training operations, neural network functions and/or architectures, or neural network use cases described herein.

Such components can be used to determine and optimize motion for an automated system to perform a determined task.

Computer Systems

FIG. **9** is a block diagram illustrating an exemplary computer system, which may be a system with interconnected devices and components, a system-on-a-chip (SOC) or some combination thereof **900** formed with a processor that may include execution units to execute an instruction, according to at least one embodiment. In at least one embodiment, computer system **900** may include, without limitation, a component, such as a processor **902** to employ execution units including logic to perform algorithms for process data, in accordance with present disclosure, such as in embodiment described herein. In at least one embodiment, computer system **900** may include processors, such as PENTIUM® Processor family, Xeon™, Itanium®, XScale™ and/or StrongARM™, Intel® Core™, or Intel® Nervana™ microprocessors available from Intel Corporation of Santa Clara, California, although other systems (including PCs having other microprocessors, engineering workstations, set-top boxes and like) may also be used. In at least one embodiment, computer system **900** may execute a version of WINDOWS® operating system available from Microsoft Corporation of Redmond, Wash., although other

operating systems (UNIX and Linux for example), embedded software, and/or graphical user interfaces, may also be used.

Embodiments may be used in other devices such as handheld devices and embedded applications. Some examples of handheld devices include cellular phones, Internet Protocol devices, digital cameras, personal digital assistants (“PDAs”), and handheld PCs. In at least one embodiment, embedded applications may include a microcontroller, a digital signal processor (“DSP”), system on a chip, network computers (“NetPCs”), set-top boxes, network hubs, wide area network (“WAN”) switches, or any other system that may perform one or more instructions in accordance with at least one embodiment.

In at least one embodiment, computer system 900 may include, without limitation, processor 902 that may include, without limitation, one or more execution units 908 to perform machine learning model training and/or inferencing according to techniques described herein. In at least one embodiment, computer system 900 is a single processor desktop or server system, but in another embodiment computer system 900 may be a multiprocessor system. In at least one embodiment, processor 902 may include, without limitation, a complex instruction set computer (“CISC”) microprocessor, a reduced instruction set computing (“RISC”) microprocessor, a very long instruction word (“VLIW”) microprocessor, a processor implementing a combination of instruction sets, or any other processor device, such as a digital signal processor, for example. In at least one embodiment, processor 902 may be coupled to a processor bus 910 that may transmit data signals between processor 902 and other components in computer system 900.

In at least one embodiment, processor 902 may include, without limitation, a Level 1 (“L1”) internal cache memory (“cache”) 904. In at least one embodiment, processor 902 may have a single internal cache or multiple levels of internal cache. In at least one embodiment, cache memory may reside external to processor 902. Other embodiments may also include a combination of both internal and external caches depending on particular implementation and needs. In at least one embodiment, register file 906 may store different types of data in various registers including, without limitation, integer registers, floating point registers, status registers, and instruction pointer register.

In at least one embodiment, execution unit 908, including, without limitation, logic to perform integer and floating point operations, also resides in processor 902. In at least one embodiment, processor 902 may also include a microcode (“ucode”) read only memory (“ROM”) that stores microcode for certain macro instructions. In at least one embodiment, execution unit 908 may include logic to handle a packed instruction set 909. In at least one embodiment, by including packed instruction set 909 in an instruction set of a general-purpose processor 902, along with associated circuitry to execute instructions, operations used by many multimedia applications may be performed using packed data in a general-purpose processor 902. In one or more embodiments, many multimedia applications may be accelerated and executed more efficiently by using full width of a processor’s data bus for performing operations on packed data, which may eliminate need to transfer smaller units of data across processor’s data bus to perform one or more operations one data element at a time.

In at least one embodiment, execution unit 908 may also be used in microcontrollers, embedded processors, graphics devices, DSPs, and other types of logic circuits. In at least one embodiment, computer system 900 may include, with-

out limitation, a memory 920. In at least one embodiment, memory 920 may be implemented as a Dynamic Random Access Memory (“DRAM”) device, a Static Random Access Memory (“SRAM”) device, flash memory device, or other memory device. In at least one embodiment, memory 920 may store instruction(s) 919 and/or data 921 represented by data signals that may be executed by processor 902.

In at least one embodiment, system logic chip may be coupled to processor bus 910 and memory 920. In at least one embodiment, system logic chip may include, without limitation, a memory controller hub (“MCH”) 916, and processor 902 may communicate with MCH 916 via processor bus 910. In at least one embodiment, MCH 916 may provide a high bandwidth memory path 918 to memory 920 for instruction and data storage and for storage of graphics commands, data and textures. In at least one embodiment, MCH 916 may direct data signals between processor 902, memory 920, and other components in computer system 900 and to bridge data signals between processor bus 910, memory 920, and a system I/O 922. In at least one embodiment, system logic chip may provide a graphics port for coupling to a graphics controller. In at least one embodiment, MCH 916 may be coupled to memory 920 through a high bandwidth memory path 918 and graphics/video card 912 may be coupled to MCH 916 through an Accelerated Graphics Port (“AGP”) interconnect 914.

In at least one embodiment, computer system 900 may use system I/O 922 that is a proprietary hub interface bus to couple MCH 916 to I/O controller hub (“ICH”) 930. In at least one embodiment, ICH 930 may provide direct connections to some I/O devices via a local I/O bus. In at least one embodiment, local I/O bus may include, without limitation, a high-speed I/O bus for connecting peripherals to memory 920, chipset, and processor 902. Examples may include, without limitation, an audio controller 929, a firmware hub (“flash BIOS”) 928, a wireless transceiver 926, a data storage 924, a legacy I/O controller 923 containing user input and keyboard interfaces 925, a serial expansion port 927, such as Universal Serial Bus (“USB”), and a network controller 934. Data storage 924 may comprise a hard disk drive, a floppy disk drive, a CD-ROM device, a flash memory device, or other mass storage device.

In at least one embodiment, FIG. 9 illustrates a system, which includes interconnected hardware devices or “chips”, whereas in other embodiments, FIG. 9 may illustrate an exemplary System on a Chip (“SoC”). In at least one embodiment, devices may be interconnected with proprietary interconnects, standardized interconnects (e.g., PCIe) or some combination thereof. In at least one embodiment, one or more components of computer system 900 are interconnected using compute express link (CXL) interconnects.

Inference and/or training logic 715 are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic 715 are provided below in conjunction with FIGS. 7A and/or 7B. In at least one embodiment, inference and/or training logic 715 may be used in system FIG. 9 for inferencing or predicting operations based, at least in part, on weight parameters calculated using neural network training operations, neural network functions and/or architectures, or neural network use cases described herein.

Such components can be used to determine and optimize motion for an automated system to perform a determined task.

FIG. 10 is a block diagram illustrating an electronic device 1000 for using a processor 1010, according to at least

one embodiment. In at least one embodiment, electronic device **1000** may be, for example and without limitation, a notebook, a tower server, a rack server, a blade server, a laptop, a desktop, a tablet, a mobile device, a phone, an embedded computer, or any other suitable electronic device.

In at least one embodiment, system **1000** may include, without limitation, processor **1010** communicatively coupled to any suitable number or kind of components, peripherals, modules, or devices. In at least one embodiment, processor **1010** coupled using a bus or interface, such as a 1° C. bus, a System Management Bus (“SMBus”), a Low Pin Count (LPC) bus, a Serial Peripheral Interface (“SPI”), a High Definition Audio (“HDA”) bus, a Serial Advance Technology Attachment (“SATA”) bus, a Universal Serial Bus (“USB”) (versions 1, 2, 3), or a Universal Asynchronous Receiver/Transmitter (“UART”) bus. In at least one embodiment, FIG. **10** illustrates a system, which includes interconnected hardware devices or “chips”, whereas in other embodiments, FIG. **10** may illustrate an exemplary System on a Chip (“SoC”). In at least one embodiment, devices illustrated in FIG. **10** may be interconnected with proprietary interconnects, standardized interconnects (e.g., PCIe) or some combination thereof. In at least one embodiment, one or more components of FIG. **10** are interconnected using compute express link (CXL) interconnects.

In at least one embodiment, FIG. **10** may include a display **1024**, a touch screen **1025**, a touch pad **1030**, a Near Field Communications unit (“NFC”) **1045**, a sensor hub **1040**, a thermal sensor **1046**, an Express Chipset (“EC”) **1035**, a Trusted Platform Module (“TPM”) **1038**, BIOS/firmware/flash memory (“BIOS, FW Flash”) **1022**, a DSP **1060**, a drive **1020** such as a Solid State Disk (“SSD”) or a Hard Disk Drive (“HDD”), a wireless local area network unit (“WLAN”) **1050**, a Bluetooth unit **1052**, a Wireless Wide Area Network unit (“WWAN”) **1056**, a Global Positioning System (GPS) **1055**, a camera (“USB 3.0 camera”) **1054** such as a USB 3.0 camera, and/or a Low Power Double Data Rate (“LPDDR”) memory unit (“LPDDR3”) **1015** implemented in, for example, LPDDR3 standard. These components may each be implemented in any suitable manner.

In at least one embodiment, other components may be communicatively coupled to processor **1010** through components discussed above. In at least one embodiment, an accelerometer **1041**, Ambient Light Sensor (“ALS”) **1042**, compass **1043**, and a gyroscope **1044** may be communicatively coupled to sensor hub **1040**. In at least one embodiment, thermal sensor **1039**, a fan **1037**, a keyboard **1036**, and a G. touch pad **1030** may be communicatively coupled to EC **1035**. In at least one embodiment, speaker **1063**, headphones **1064**, and microphone (“mic”) **1065** may be communicatively coupled to an audio unit (“audio codec and class d amp”) **1062**, which may in turn be communicatively coupled to DSP **1060**. In at least one embodiment, audio unit **1064** may include, for example and without limitation, an audio coder/decoder (“codec”) and a class D amplifier. In at least one embodiment, SIM card (“SIM”) **1057** may be communicatively coupled to WWAN unit **1056**. In at least one embodiment, components such as WLAN unit **1050** and Bluetooth unit **1052**, as well as WWAN unit **1056** may be implemented in a Next Generation Form Factor (“NGFF”).

Inference and/or training logic **715** are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic **715** are provided below in conjunction with FIGS. **7a** and/or **7b**. In at least one embodiment, inference and/or training logic **715** may be used in system FIG. **10** for

inferencing or predicting operations based, at least in part, on weight parameters calculated using neural network training operations, neural network functions and/or architectures, or neural network use cases described herein.

Such components can be used to determine and optimize motion for an automated system to perform a determined task.

FIG. **11** is a block diagram of a processing system, according to at least one embodiment. In at least one embodiment, system **1100** includes one or more processors **1102** and one or more graphics processors **1108**, and may be a single processor desktop system, a multiprocessor workstation system, or a server system having a large number of processors **1102** or processor cores **1107**. In at least one embodiment, system **1100** is a processing platform incorporated within a system-on-a-chip (SoC) integrated circuit for use in mobile, handheld, or embedded devices.

In at least one embodiment, system **1100** can include, or be incorporated within a server-based gaming platform, a game console, including a game and media console, a mobile gaming console, a handheld game console, or an online game console. In at least one embodiment, system **1100** is a mobile phone, smart phone, tablet computing device or mobile Internet device. In at least one embodiment, processing system **1100** can also include, couple with, or be integrated within a wearable device, such as a smart watch wearable device, smart eyewear device, augmented reality device, or virtual reality device. In at least one embodiment, processing system **1100** is a television or set top box device having one or more processors **1102** and a graphical interface generated by one or more graphics processors **1108**.

In at least one embodiment, one or more processors **1102** each include one or more processor cores **1107** to process instructions which, when executed, perform operations for system and user software. In at least one embodiment, each of one or more processor cores **1107** is configured to process a specific instruction set **1109**. In at least one embodiment, instruction set **1109** may facilitate Complex Instruction Set Computing (CISC), Reduced Instruction Set Computing (RISC), or computing via a Very Long Instruction Word (VLIW). In at least one embodiment, processor cores **1107** may each process a different instruction set **1109**, which may include instructions to facilitate emulation of other instruction sets. In at least one embodiment, processor core **1107** may also include other processing devices, such as a Digital Signal Processor (DSP).

In at least one embodiment, processor **1102** includes cache memory **1104**. In at least one embodiment, processor **1102** can have a single internal cache or multiple levels of internal cache. In at least one embodiment, cache memory is shared among various components of processor **1102**. In at least one embodiment, processor **1102** also uses an external cache (e.g., a Level-3 (L3) cache or Last Level Cache (LLC)) (not shown), which may be shared among processor cores **1107** using known cache coherency techniques. In at least one embodiment, register file **1106** is additionally included in processor **1102** which may include different types of registers for storing different types of data (e.g., integer registers, floating point registers, status registers, and an instruction pointer register). In at least one embodiment, register file **1106** may include general-purpose registers or other registers.

In at least one embodiment, one or more processor(s) **1102** are coupled with one or more interface bus(es) **1110** to transmit communication signals such as address, data, or control signals between processor **1102** and other compo-

nents in system 1100. In at least one embodiment, interface bus 1110, in one embodiment, can be a processor bus, such as a version of a Direct Media Interface (DMI) bus. In at least one embodiment, interface 1110 is not limited to a DMI bus, and may include one or more Peripheral Component Interconnect buses (e.g., PCI, PCI Express), memory busses, or other types of interface busses. In at least one embodiment processor(s) 1102 include an integrated memory controller 1116 and a platform controller hub 1130. In at least one embodiment, memory controller 1116 facilitates communication between a memory device and other components of system 1100, while platform controller hub (PCH) 1130 provides connections to I/O devices via a local I/O bus.

In at least one embodiment, memory device 1120 can be a dynamic random access memory (DRAM) device, a static random access memory (SRAM) device, flash memory device, phase-change memory device, or some other memory device having suitable performance to serve as process memory. In at least one embodiment memory device 1120 can operate as system memory for system 1100, to store data 1122 and instructions 1121 for use when one or more processors 1102 executes an application or process. In at least one embodiment, memory controller 1116 also couples with an optional external graphics processor 1112, which may communicate with one or more graphics processors 1108 in processors 1102 to perform graphics and media operations. In at least one embodiment, a display device 1111 can connect to processor(s) 1102. In at least one embodiment display device 1111 can include one or more of an internal display device, as in a mobile electronic device or a laptop device or an external display device attached via a display interface (e.g., DisplayPort, etc.). In at least one embodiment, display device 1111 can include a head mounted display (HMD) such as a stereoscopic display device for use in virtual reality (VR) applications or augmented reality (AR) applications.

In at least one embodiment, platform controller hub 1130 enables peripherals to connect to memory device 1120 and processor 1102 via a high-speed I/O bus. In at least one embodiment, I/O peripherals include, but are not limited to, an audio controller 1146, a network controller 1134, a firmware interface 1128, a wireless transceiver 1126, touch sensors 1125, a data storage device 1124 (e.g., hard disk drive, flash memory, etc.). In at least one embodiment, data storage device 1124 can connect via a storage interface (e.g., SATA) or via a peripheral bus, such as a Peripheral Component Interconnect bus (e.g., PCI, PCI Express). In at least one embodiment, touch sensors 1125 can include touch screen sensors, pressure sensors, or fingerprint sensors. In at least one embodiment, wireless transceiver 1126 can be a Wi-Fi transceiver, a Bluetooth transceiver, or a mobile network transceiver such as a 3G, 4G, or Long Term Evolution (LTE) transceiver. In at least one embodiment, firmware interface 1128 enables communication with system firmware, and can be, for example, a unified extensible firmware interface (UEFI). In at least one embodiment, network controller 1134 can enable a network connection to a wired network. In at least one embodiment, a high-performance network controller (not shown) couples with interface bus 1110. In at least one embodiment, audio controller 1146 is a multi-channel high definition audio controller. In at least one embodiment, system 1100 includes an optional legacy I/O controller 1140 for coupling legacy (e.g., Personal System 2 (PS/2)) devices to system. In at least one embodiment, platform controller hub 1130 can also connect to one or more Universal Serial Bus (USB) con-

trollers 1142 connect input devices, such as keyboard and mouse 1143 combinations, a camera 1144, or other USB input devices.

In at least one embodiment, an instance of memory controller 1116 and platform controller hub 1130 may be integrated into a discreet external graphics processor, such as external graphics processor 1112. In at least one embodiment, platform controller hub 1130 and/or memory controller 1116 may be external to one or more processor(s) 1102. For example, in at least one embodiment, system 1100 can include an external memory controller 1116 and platform controller hub 1130, which may be configured as a memory controller hub and peripheral controller hub within a system chipset that is in communication with processor(s) 1102.

Inference and/or training logic 715 are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic 715 are provided below in conjunction with FIGS. 7A and/or 7B. In at least one embodiment portions or all of inference and/or training logic 715 may be incorporated into graphics processor 1500. For example, in at least one embodiment, training and/or inferencing techniques described herein may use one or more of ALUs embodied in a graphics processor. Moreover, in at least one embodiment, inferencing and/or training operations described herein may be done using logic other than logic illustrated in FIG. 7A or 7B. In at least one embodiment, weight parameters may be stored in on-chip or off-chip memory and/or registers (shown or not shown) that configure ALUs of a graphics processor to perform one or more machine learning algorithms, neural network architectures, use cases, or training techniques described herein.

Such components can be used to determine and optimize motion for an automated system to perform a determined task.

FIG. 12 is a block diagram of a processor 1200 having one or more processor cores 1202A-1202N, an integrated memory controller 1214, and an integrated graphics processor 1208, according to at least one embodiment. In at least one embodiment, processor 1200 can include additional cores up to and including additional core 1202N represented by dashed lined boxes. In at least one embodiment, each of processor cores 1202A-1202N includes one or more internal cache units 1204A-1204N. In at least one embodiment, each processor core also has access to one or more shared cached units 1206.

In at least one embodiment, internal cache units 1204A-1204N and shared cache units 1206 represent a cache memory hierarchy within processor 1200. In at least one embodiment, cache memory units 1204A-1204N may include at least one level of instruction and data cache within each processor core and one or more levels of shared mid-level cache, such as a Level 2 (L2), Level 3 (L3), Level 4 (L4), or other levels of cache, where a highest level of cache before external memory is classified as an LLC. In at least one embodiment, cache coherency logic maintains coherency between various cache units 1206 and 1204A-1204N.

In at least one embodiment, processor 1200 may also include a set of one or more bus controller units 1216 and a system agent core 1210. In at least one embodiment, one or more bus controller units 1216 manage a set of peripheral buses, such as one or more PCI or PCI express busses. In at least one embodiment, system agent core 1210 provides management functionality for various processor components. In at least one embodiment, system agent core 1210

includes one or more integrated memory controllers **1214** to manage access to various external memory devices (not shown).

In at least one embodiment, one or more of processor cores **1202A-1202N** include support for simultaneous multi-threading. In at least one embodiment, system agent core **1210** includes components for coordinating and operating cores **1202A-1202N** during multi-threaded processing. In at least one embodiment, system agent core **1210** may additionally include a power control unit (PCU), which includes logic and components to regulate one or more power states of processor cores **1202A-1202N** and graphics processor **1208**.

In at least one embodiment, processor **1200** additionally includes graphics processor **1208** to execute graphics processing operations. In at least one embodiment, graphics processor **1208** couples with shared cache units **1206**, and system agent core **1210**, including one or more integrated memory controllers **1214**. In at least one embodiment, system agent core **1210** also includes a display controller **1211** to drive graphics processor output to one or more coupled displays. In at least one embodiment, display controller **1211** may also be a separate module coupled with graphics processor **1208** via at least one interconnect, or may be integrated within graphics processor **1208**.

In at least one embodiment, a ring based interconnect unit **1212** is used to couple internal components of processor **1200**. In at least one embodiment, an alternative interconnect unit may be used, such as a point-to-point interconnect, a switched interconnect, or other techniques. In at least one embodiment, graphics processor **1208** couples with ring interconnect **1212** via an I/O link **1213**.

In at least one embodiment, I/O link **1213** represents at least one of multiple varieties of I/O interconnects, including an on package I/O interconnect which facilitates communication between various processor components and a high-performance embedded memory module **1218**, such as an eDRAM module. In at least one embodiment, each of processor cores **1202A-1202N** and graphics processor **1208** use embedded memory modules **1218** as a shared Last Level Cache.

In at least one embodiment, processor cores **1202A-1202N** are homogenous cores executing a common instruction set architecture. In at least one embodiment, processor cores **1202A-1202N** are heterogeneous in terms of instruction set architecture (ISA), where one or more of processor cores **1202A-1202N** execute a common instruction set, while one or more other cores of processor cores **1202A-1202N** executes a subset of a common instruction set or a different instruction set. In at least one embodiment, processor cores **1202A-1202N** are heterogeneous in terms of microarchitecture, where one or more cores having a relatively higher power consumption couple with one or more power cores having a lower power consumption. In at least one embodiment, processor **1200** can be implemented on one or more chips or as an SoC integrated circuit.

Inference and/or training logic **715** are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic **715** are provided below in conjunction with FIGS. **7a** and/or **7b**. In at least one embodiment portions or all of inference and/or training logic **715** may be incorporated into processor **1200**. For example, in at least one embodiment, training and/or inferencing techniques described herein may use one or more of ALUs embodied in graphics processor **1512**, graphics core(s) **1202A-1202N**, or other components in FIG. **12**. Moreover, in at least one embodiment, inferenc-

ing and/or training operations described herein may be done using logic other than logic illustrated in FIG. **7A** or **7B**. In at least one embodiment, weight parameters may be stored in on-chip or off-chip memory and/or registers (shown or not shown) that configure ALUs of graphics processor **1200** to perform one or more machine learning algorithms, neural network architectures, use cases, or training techniques described herein.

Such components can be used to determine and optimize motion for an automated system to perform a determined task.

Virtualized Computing Platform

FIG. **13** is an example data flow diagram for a process **1300** of generating and deploying an image processing and inferencing pipeline, in accordance with at least one embodiment. In at least one embodiment, process **1300** may be deployed for use with imaging devices, processing devices, and/or other device types at one or more facilities **1302**. Process **1300** may be executed within a training system **1304** and/or a deployment system **1306**. In at least one embodiment, training system **1304** may be used to perform training, deployment, and implementation of machine learning models (e.g., neural networks, object detection algorithms, computer vision algorithms, etc.) for use in deployment system **1306**. In at least one embodiment, deployment system **1306** may be configured to offload processing and compute resources among a distributed computing environment to reduce infrastructure requirements at facility **1302**. In at least one embodiment, one or more applications in a pipeline may use or call upon services (e.g., inference, visualization, compute, AI, etc.) of deployment system **1306** during execution of applications.

In at least one embodiment, some of applications used in advanced processing and inferencing pipelines may use machine learning models or other AI to perform one or more processing steps. In at least one embodiment, machine learning models may be trained at facility **1302** using data **1308** (such as imaging data) generated at facility **1302** (and stored on one or more picture archiving and communication system (PACS) servers at facility **1302**), may be trained using imaging or sequencing data **1308** from another facility(ies), or a combination thereof. In at least one embodiment, training system **1304** may be used to provide applications, services, and/or other resources for generating working, deployable machine learning models for deployment system **1306**.

In at least one embodiment, model registry **1324** may be backed by object storage that may support versioning and object metadata. In at least one embodiment, object storage may be accessible through, for example, a cloud storage (e.g., cloud **1426** of FIG. **14**) compatible application programming interface (API) from within a cloud platform. In at least one embodiment, machine learning models within model registry **1324** may uploaded, listed, modified, or deleted by developers or partners of a system interacting with an API. In at least one embodiment, an API may provide access to methods that allow users with appropriate credentials to associate models with applications, such that models may be executed as part of execution of containerized instantiations of applications.

In at least one embodiment, training pipeline **1404** (FIG. **14**) may include a scenario where facility **1302** is training their own machine learning model, or has an existing machine learning model that needs to be optimized or updated. In at least one embodiment, imaging data **1308**

generated by imaging device(s), sequencing devices, and/or other device types may be received. In at least one embodiment, once imaging data **1308** is received, AI-assisted annotation **1310** may be used to aid in generating annotations corresponding to imaging data **1308** to be used as ground truth data for a machine learning model. In at least one embodiment, AI-assisted annotation **1310** may include one or more machine learning models (e.g., convolutional neural networks (CNNs)) that may be trained to generate annotations corresponding to certain types of imaging data **1308** (e.g., from certain devices). In at least one embodiment, AI-assisted annotations **1310** may then be used directly, or may be adjusted or fine-tuned using an annotation tool to generate ground truth data. In at least one embodiment, AI-assisted annotations **1310**, labeled clinic data **1312**, or a combination thereof may be used as ground truth data for training a machine learning model. In at least one embodiment, a trained machine learning model may be referred to as output model **1316**, and may be used by deployment system **1306**, as described herein.

In at least one embodiment, training pipeline **1404** (FIG. **14**) may include a scenario where facility **1302** needs a machine learning model for use in performing one or more processing tasks for one or more applications in deployment system **1306**, but facility **1302** may not currently have such a machine learning model (or may not have a model that is optimized, efficient, or effective for such purposes). In at least one embodiment, an existing machine learning model may be selected from a model registry **1324**. In at least one embodiment, model registry **1324** may include machine learning models trained to perform a variety of different inference tasks on imaging data. In at least one embodiment, machine learning models in model registry **1324** may have been trained on imaging data from different facilities than facility **1302** (e.g., facilities remotely located). In at least one embodiment, machine learning models may have been trained on imaging data from one location, two locations, or any number of locations. In at least one embodiment, when being trained on imaging data from a specific location, training may take place at that location, or at least in a manner that protects confidentiality of imaging data or restricts imaging data from being transferred off-premises. In at least one embodiment, once a model is trained—or partially trained—at one location, a machine learning model may be added to model registry **1324**. In at least one embodiment, a machine learning model may then be retrained, or updated, at any number of other facilities, and a retrained or updated model may be made available in model registry **1324**. In at least one embodiment, a machine learning model may then be selected from model registry **1324**—and referred to as output model **1316**—and may be used in deployment system **1306** to perform one or more processing tasks for one or more applications of a deployment system.

In at least one embodiment, training pipeline **1404** (FIG. **14**), a scenario may include facility **1302** requiring a machine learning model for use in performing one or more processing tasks for one or more applications in deployment system **1306**, but facility **1302** may not currently have such a machine learning model (or may not have a model that is optimized, efficient, or effective for such purposes). In at least one embodiment, a machine learning model selected from model registry **1324** may not be fine-tuned or optimized for imaging data **1308** generated at facility **1302** because of differences in populations, robustness of training data used to train a machine learning model, diversity in anomalies of training data, and/or other issues with training

data. In at least one embodiment, AI-assisted annotation **1310** may be used to aid in generating annotations corresponding to imaging data **1308** to be used as ground truth data for retraining or updating a machine learning model. In at least one embodiment, labeled data **1312** may be used as ground truth data for training a machine learning model. In at least one embodiment, retraining or updating a machine learning model may be referred to as model training **1314**. In at least one embodiment, model training **1314**—e.g., AI-assisted annotations **1310**, labeled clinic data **1312**, or a combination thereof—may be used as ground truth data for retraining or updating a machine learning model. In at least one embodiment, a trained machine learning model may be referred to as output model **1316**, and may be used by deployment system **1306**, as described herein.

In at least one embodiment, deployment system **1306** may include software **1318**, services **1320**, hardware **1322**, and/or other components, features, and functionality. In at least one embodiment, deployment system **1306** may include a software “stack,” such that software **1318** may be built on top of services **1320** and may use services **1320** to perform some or all of processing tasks, and services **1320** and software **1318** may be built on top of hardware **1322** and use hardware **1322** to execute processing, storage, and/or other compute tasks of deployment system **1306**. In at least one embodiment, software **1318** may include any number of different containers, where each container may execute an instantiation of an application. In at least one embodiment, each application may perform one or more processing tasks in an advanced processing and inferencing pipeline (e.g., inferencing, object detection, feature detection, segmentation, image enhancement, calibration, etc.). In at least one embodiment, an advanced processing and inferencing pipeline may be defined based on selections of different containers that are desired or required for processing imaging data **1308**, in addition to containers that receive and configure imaging data for use by each container and/or for use by facility **1302** after processing through a pipeline (e.g., to convert outputs back to a usable data type). In at least one embodiment, a combination of containers within software **1318** (e.g., that make up a pipeline) may be referred to as a virtual instrument (as described in more detail herein), and a virtual instrument may leverage services **1320** and hardware **1322** to execute some or all processing tasks of applications instantiated in containers.

In at least one embodiment, a data processing pipeline may receive input data (e.g., imaging data **1308**) in a specific format in response to an inference request (e.g., a request from a user of deployment system **1306**). In at least one embodiment, input data may be representative of one or more images, video, and/or other data representations generated by one or more imaging devices. In at least one embodiment, data may undergo pre-processing as part of data processing pipeline to prepare data for processing by one or more applications. In at least one embodiment, post-processing may be performed on an output of one or more inferencing tasks or other processing tasks of a pipeline to prepare an output data for a next application and/or to prepare output data for transmission and/or use by a user (e.g., as a response to an inference request). In at least one embodiment, inferencing tasks may be performed by one or more machine learning models, such as trained or deployed neural networks, which may include output models **1316** of training system **1304**.

In at least one embodiment, tasks of data processing pipeline may be encapsulated in a container(s) that each represents a discrete, fully functional instantiation of an

application and virtualized computing environment that is able to reference machine learning models. In at least one embodiment, containers or applications may be published into a private (e.g., limited access) area of a container registry (described in more detail herein), and trained or deployed models may be stored in model registry **1324** and associated with one or more applications. In at least one embodiment, images of applications (e.g., container images) may be available in a container registry, and once selected by a user from a container registry for deployment in a pipeline, an image may be used to generate a container for an instantiation of an application for use by a user's system.

In at least one embodiment, developers (e.g., software developers, clinicians, doctors, etc.) may develop, publish, and store applications (e.g., as containers) for performing image processing and/or inferencing on supplied data. In at least one embodiment, development, publishing, and/or storing may be performed using a software development kit (SDK) associated with a system (e.g., to ensure that an application and/or container developed is compliant with or compatible with a system). In at least one embodiment, an application that is developed may be tested locally (e.g., at a first facility, on data from a first facility) with an SDK which may support at least some of services **1320** as a system (e.g., system **1400** of FIG. **14**). In at least one embodiment, because DICOM objects may contain anywhere from one to hundreds of images or other data types, and due to a variation in data, a developer may be responsible for managing (e.g., setting constructs for, building pre-processing into an application, etc.) extraction and preparation of incoming data. In at least one embodiment, once validated by system **1400** (e.g., for accuracy), an application may be available in a container registry for selection and/or implementation by a user to perform one or more processing tasks with respect to data at a facility (e.g., a second facility) of a user.

In at least one embodiment, developers may then share applications or containers through a network for access and use by users of a system (e.g., system **1400** of FIG. **14**). In at least one embodiment, completed and validated applications or containers may be stored in a container registry and associated machine learning models may be stored in model registry **1324**. In at least one embodiment, a requesting entity—who provides an inference or image processing request—may browse a container registry and/or model registry **1324** for an application, container, dataset, machine learning model, etc., select a desired combination of elements for inclusion in data processing pipeline, and submit an imaging processing request. In at least one embodiment, a request may include input data (and associated patient data, in some examples) that is necessary to perform a request, and/or may include a selection of application(s) and/or machine learning models to be executed in processing a request. In at least one embodiment, a request may then be passed to one or more components of deployment system **1306** (e.g., a cloud) to perform processing of data processing pipeline. In at least one embodiment, processing by deployment system **1306** may include referencing selected elements (e.g., applications, containers, models, etc.) from a container registry and/or model registry **1324**. In at least one embodiment, once results are generated by a pipeline, results may be returned to a user for reference (e.g., for viewing in a viewing application suite executing on a local, on-premises workstation or terminal).

In at least one embodiment, to aid in processing or execution of applications or containers in pipelines, services **1320** may be leveraged. In at least one embodiment, services

1320 may include compute services, artificial intelligence (AI) services, visualization services, and/or other service types. In at least one embodiment, services **1320** may provide functionality that is common to one or more applications in software **1318**, so functionality may be abstracted to a service that may be called upon or leveraged by applications. In at least one embodiment, functionality provided by services **1320** may run dynamically and more efficiently, while also scaling well by allowing applications to process data in parallel (e.g., using a parallel computing platform **1430** (FIG. **14**)). In at least one embodiment, rather than each application that shares a same functionality offered by a service **1320** being required to have a respective instance of service **1320**, service **1320** may be shared between and among various applications. In at least one embodiment, services may include an inference server or engine that may be used for executing detection or segmentation tasks, as non-limiting examples. In at least one embodiment, a model training service may be included that may provide machine learning model training and/or retraining capabilities. In at least one embodiment, a data augmentation service may further be included that may provide GPU accelerated data (e.g., DICOM, RIS, CIS, REST compliant, RPC, raw, etc.) extraction, resizing, scaling, and/or other augmentation. In at least one embodiment, a visualization service may be used that may add image rendering effects—such as ray-tracing, rasterization, denoising, sharpening, etc.—to add realism to two-dimensional (2D) and/or three-dimensional (3D) models. In at least one embodiment, virtual instrument services may be included that provide for beam-forming, segmentation, inferencing, imaging, and/or support for other applications within pipelines of virtual instruments.

In at least one embodiment, where a service **1320** includes an AI service (e.g., an inference service), one or more machine learning models may be executed by calling upon (e.g., as an API call) an inference service (e.g., an inference server) to execute machine learning model(s), or processing thereof, as part of application execution. In at least one embodiment, where another application includes one or more machine learning models for segmentation tasks, an application may call upon an inference service to execute machine learning models for performing one or more of processing operations associated with segmentation tasks. In at least one embodiment, software **1318** implementing advanced processing and inferencing pipeline that includes segmentation application and anomaly detection application may be streamlined because each application may call upon a same inference service to perform one or more inferencing tasks.

In at least one embodiment, hardware **1322** may include GPUs, CPUs, graphics cards, an AI/deep learning system (e.g., an AI supercomputer, such as NVIDIA's DGX), a cloud platform, or a combination thereof. In at least one embodiment, different types of hardware **1322** may be used to provide efficient, purpose-built support for software **1318** and services **1320** in deployment system **1306**. In at least one embodiment, use of GPU processing may be implemented for processing locally (e.g., at facility **1302**), within an AI/deep learning system, in a cloud system, and/or in other processing components of deployment system **1306** to improve efficiency, accuracy, and efficacy of image processing and generation. In at least one embodiment, software **1318** and/or services **1320** may be optimized for GPU processing with respect to deep learning, machine learning, and/or high-performance computing, as non-limiting examples. In at least one embodiment, at least some of

computing environment of deployment system **1306** and/or training system **1304** may be executed in a datacenter one or more supercomputers or high performance computing systems, with GPU optimized software (e.g., hardware and software combination of NVIDIA's DGX System). In at least one embodiment, hardware **1322** may include any number of GPUs that may be called upon to perform processing of data in parallel, as described herein. In at least one embodiment, cloud platform may further include GPU processing for GPU-optimized execution of deep learning tasks, machine learning tasks, or other computing tasks. In at least one embodiment, cloud platform (e.g., NVIDIA's NGC) may be executed using an AI/deep learning super-computer(s) and/or GPU-optimized software (e.g., as provided on NVIDIA's DGX Systems) as a hardware abstraction and scaling platform. In at least one embodiment, cloud platform may integrate an application container clustering system or orchestration system (e.g., KUBERNETES) on multiple GPUs to enable seamless scaling and load balancing.

FIG. **14** is a system diagram for an example system **1400** for generating and deploying an imaging deployment pipeline, in accordance with at least one embodiment. In at least one embodiment, system **1400** may be used to implement process **1300** of FIG. **13** and/or other processes including advanced processing and inferencing pipelines. In at least one embodiment, system **1400** may include training system **1304** and deployment system **1306**. In at least one embodiment, training system **1304** and deployment system **1306** may be implemented using software **1318**, services **1320**, and/or hardware **1322**, as described herein.

In at least one embodiment, system **1400** (e.g., training system **1304** and/or deployment system **1306**) may be implemented in a cloud computing environment (e.g., using cloud **1426**). In at least one embodiment, system **1400** may be implemented locally with respect to a healthcare services facility, or as a combination of both cloud and local computing resources. In at least one embodiment, access to APIs in cloud **1426** may be restricted to authorized users through enacted security measures or protocols. In at least one embodiment, a security protocol may include web tokens that may be signed by an authentication (e.g., AuthN, AuthZ, Gluecon, etc.) service and may carry appropriate authorization. In at least one embodiment, APIs of virtual instruments (described herein), or other instantiations of system **1400**, may be restricted to a set of public IPs that have been vetted or authorized for interaction.

In at least one embodiment, various components of system **1400** may communicate between and among one another using any of a variety of different network types, including but not limited to local area networks (LANs) and/or wide area networks (WANs) via wired and/or wireless communication protocols. In at least one embodiment, communication between facilities and components of system **1400** (e.g., for transmitting inference requests, for receiving results of inference requests, etc.) may be communicated over data bus(es), wireless data protocols (Wi-Fi), wired data protocols (e.g., Ethernet), etc.

In at least one embodiment, training system **1304** may execute training pipelines **1404**, similar to those described herein with respect to FIG. **13**. In at least one embodiment, where one or more machine learning models are to be used in deployment pipelines **1410** by deployment system **1306**, training pipelines **1404** may be used to train or retrain one or more (e.g. pre-trained) models, and/or implement one or more of pre-trained models **1406** (e.g., without a need for retraining or updating). In at least one embodiment, as a

result of training pipelines **1404**, output model(s) **1316** may be generated. In at least one embodiment, training pipelines **1404** may include any number of processing steps, such as but not limited to imaging data (or other input data) conversion or adaption. In at least one embodiment, for different machine learning models used by deployment system **1306**, different training pipelines **1404** may be used. In at least one embodiment, training pipeline **1404** similar to a first example described with respect to FIG. **13** may be used for a first machine learning model, training pipeline **1404** similar to a second example described with respect to FIG. **13** may be used for a second machine learning model, and training pipeline **1404** similar to a third example described with respect to FIG. **13** may be used for a third machine learning model. In at least one embodiment, any combination of tasks within training system **1304** may be used depending on what is required for each respective machine learning model. In at least one embodiment, one or more of machine learning models may already be trained and ready for deployment so machine learning models may not undergo any processing by training system **1304**, and may be implemented by deployment system **1306**.

In at least one embodiment, output model(s) **1316** and/or pre-trained model(s) **1406** may include any types of machine learning models depending on implementation or embodiment. In at least one embodiment, and without limitation, machine learning models used by system **1400** may include machine learning model(s) using linear regression, logistic regression, decision trees, support vector machines (SVM), Naïve Bayes, k-nearest neighbor (Knn), K means clustering, random forest, dimensionality reduction algorithms, gradient boosting algorithms, neural networks (e.g., auto-encoders, convolutional, recurrent, perceptrons, Long/Short Term Memory (LSTM), Hopfield, Boltzmann, deep belief, deconvolutional, generative adversarial, liquid state machine, etc.), and/or other types of machine learning models.

In at least one embodiment, training pipelines **1404** may include AI-assisted annotation, as described in more detail herein with respect to at least FIG. **15B**. In at least one embodiment, labeled data **1312** (e.g., traditional annotation) may be generated by any number of techniques. In at least one embodiment, labels or other annotations may be generated within a drawing program (e.g., an annotation program), a computer aided design (CAD) program, a labeling program, another type of program suitable for generating annotations or labels for ground truth, and/or may be hand drawn, in some examples. In at least one embodiment, ground truth data may be synthetically produced (e.g., generated from computer models or renderings), real produced (e.g., designed and produced from real-world data), machine-automated (e.g., using feature analysis and learning to extract features from data and then generate labels), human annotated (e.g., labeler, or annotation expert, defines location of labels), and/or a combination thereof. In at least one embodiment, for each instance of imaging data **1308** (or other data type used by machine learning models), there may be corresponding ground truth data generated by training system **1304**. In at least one embodiment, AI-assisted annotation may be performed as part of deployment pipelines **1410**; either in addition to, or in lieu of AI-assisted annotation included in training pipelines **1404**. In at least one embodiment, system **1400** may include a multi-layer platform that may include a software layer (e.g., software **1318**) of diagnostic applications (or other application types) that may perform one or more medical imaging and diagnostic functions. In at least one embodiment, system **1400** may be communicatively coupled to (e.g., via encrypted links)

PACS server networks of one or more facilities. In at least one embodiment, system **1400** may be configured to access and referenced data from PACS servers to perform operations, such as training machine learning models, deploying machine learning models, image processing, inferencing, and/or other operations.

In at least one embodiment, a software layer may be implemented as a secure, encrypted, and/or authenticated API through which applications or containers may be invoked (e.g., called) from an external environment(s) (e.g., facility **1302**). In at least one embodiment, applications may then call or execute one or more services **1320** for performing compute, AI, or visualization tasks associated with respective applications, and software **1318** and/or services **1320** may leverage hardware **1322** to perform processing tasks in an effective and efficient manner. In at least one embodiment, communications sent to, or received by, a training system **1304** and a deployment system **1306** may occur using a pair of DICOM adapters **1402A**, **1402B**.

In at least one embodiment, deployment system **1306** may execute deployment pipelines **1410**. In at least one embodiment, deployment pipelines **1410** may include any number of applications that may be sequentially, non-sequentially, or otherwise applied to imaging data (and/or other data types) generated by imaging devices, sequencing devices, genomics devices, etc.—including AI-assisted annotation, as described above. In at least one embodiment, as described herein, a deployment pipeline **1410** for an individual device may be referred to as a virtual instrument for a device (e.g., a virtual ultrasound instrument, a virtual CT scan instrument, a virtual sequencing instrument, etc.). In at least one embodiment, for a single device, there may be more than one deployment pipeline **1410** depending on information desired from data generated by a device. In at least one embodiment, where detections of anomalies are desired from an MRI machine, there may be a first deployment pipeline **1410**, and where image enhancement is desired from output of an MRI machine, there may be a second deployment pipeline **1410**.

In at least one embodiment, an image generation application may include a processing task that includes use of a machine learning model. In at least one embodiment, a user may desire to use their own machine learning model, or to select a machine learning model from model registry **1324**. In at least one embodiment, a user may implement their own machine learning model or select a machine learning model for inclusion in an application for performing a processing task. In at least one embodiment, applications may be selectable and customizable, and by defining constructs of applications, deployment and implementation of applications for a particular user are presented as a more seamless user experience. In at least one embodiment, by leveraging other features of system **1400**—such as services **1320** and hardware **1322**—deployment pipelines **1410** may be even more user friendly, provide for easier integration, and produce more accurate, efficient, and timely results.

In at least one embodiment, deployment system **1306** may include a user interface **1414** (e.g., a graphical user interface, a web interface, etc.) that may be used to select applications for inclusion in deployment pipeline(s) **1410**, arrange applications, modify or change applications or parameters or constructs thereof, use and interact with deployment pipeline(s) **1410** during set-up and/or deployment, and/or to otherwise interact with deployment system **1306**. In at least one embodiment, although not illustrated with respect to training system **1304**, user interface **1414** (or a different user interface) may be used for selecting models for use in deployment system **1306**, for selecting models for training,

or retraining, in training system **1304**, and/or for otherwise interacting with training system **1304**.

In at least one embodiment, pipeline manager **1412** may be used, in addition to an application orchestration system **1428**, to manage interaction between applications or containers of deployment pipeline(s) **1410** and services **1320** and/or hardware **1322**. In at least one embodiment, pipeline manager **1412** may be configured to facilitate interactions from application to application, from application to service **1320**, and/or from application or service to hardware **1322**. In at least one embodiment, although illustrated as included in software **1318**, this is not intended to be limiting, and in some examples (e.g., as illustrated in FIG. **12cc**) pipeline manager **1412** may be included in services **1320**. In at least one embodiment, application orchestration system **1428** (e.g., Kubernetes, DOCKER, etc.) may include a container orchestration system that may group applications into containers as logical units for coordination, management, scaling, and deployment. In at least one embodiment, by associating applications from deployment pipeline(s) **1410** (e.g., a reconstruction application, a segmentation application, etc.) with individual containers, each application may execute in a self-contained environment (e.g., at a kernel level) to increase speed and efficiency.

In at least one embodiment, each application and/or container (or image thereof) may be individually developed, modified, and deployed (e.g., a first user or developer may develop, modify, and deploy a first application and a second user or developer may develop, modify, and deploy a second application separate from a first user or developer), which may allow for focus on, and attention to, a task of a single application and/or container(s) without being hindered by tasks of another application(s) or container(s). In at least one embodiment, communication, and cooperation between different containers or applications may be aided by pipeline manager **1412** and application orchestration system **1428**. In at least one embodiment, so long as an expected input and/or output of each container or application is known by a system (e.g., based on constructs of applications or containers), application orchestration system **1428** and/or pipeline manager **1412** may facilitate communication among and between, and sharing of resources among and between, each of applications or containers. In at least one embodiment, because one or more of applications or containers in deployment pipeline(s) **1410** may share same services and resources, application orchestration system **1428** may orchestrate, load balance, and determine sharing of services or resources between and among various applications or containers. In at least one embodiment, a scheduler may be used to track resource requirements of applications or containers, current usage or planned usage of these resources, and resource availability. In at least one embodiment, a scheduler may thus allocate resources to different applications and distribute resources between and among applications in view of requirements and availability of a system. In some examples, a scheduler (and/or other component of application orchestration system **1428**) may determine resource availability and distribution based on constraints imposed on a system (e.g., user constraints), such as quality of service (QoS), urgency of need for data outputs (e.g., to determine whether to execute real-time processing or delayed processing), etc.

In at least one embodiment, services **1320** leveraged by and shared by applications or containers in deployment system **1306** may include compute services **1416**, AI services **1418**, visualization services **1420**, and/or other service types. In at least one embodiment, applications may call

(e.g., execute) one or more of services **1320** to perform processing operations for an application. In at least one embodiment, compute services **1416** may be leveraged by applications to perform super-computing or other high-performance computing (HPC) tasks. In at least one embodiment, compute service(s) **1416** may be leveraged to perform parallel processing (e.g., using a parallel computing platform **1430**) for processing data through one or more of applications and/or one or more tasks of a single application, substantially simultaneously. In at least one embodiment, parallel computing platform **1430** (e.g., NVIDIA's CUDA) may enable general purpose computing on GPUs (GPGPU) (e.g., GPUs **1422**). In at least one embodiment, a software layer of parallel computing platform **1430** may provide access to virtual instruction sets and parallel computational elements of GPUs, for execution of compute kernels. In at least one embodiment, parallel computing platform **1430** may include memory and, in some embodiments, a memory may be shared between and among multiple containers, and/or between and among different processing tasks within a single container. In at least one embodiment, inter-process communication (IPC) calls may be generated for multiple containers and/or for multiple processes within a container to use same data from a shared segment of memory of parallel computing platform **1430** (e.g., where multiple different stages of an application or multiple applications are processing same information). In at least one embodiment, rather than making a copy of data and moving data to different locations in memory (e.g., a read/write operation), same data in same location of a memory may be used for any number of processing tasks (e.g., at a same time, at different times, etc.). In at least one embodiment, as data is used to generate new data as a result of processing, this information of a new location of data may be stored and shared between various applications. In at least one embodiment, location of data and a location of updated or modified data may be part of a definition of how a payload is understood within containers.

In at least one embodiment, AI services **1418** may be leveraged to perform inferencing services for executing machine learning model(s) associated with applications (e.g., tasked with performing one or more processing tasks of an application). In at least one embodiment, AI services **1418** may leverage AI system **1424** to execute machine learning model(s) (e.g., neural networks, such as CNNs) for segmentation, reconstruction, object detection, feature detection, classification, and/or other inferencing tasks. In at least one embodiment, applications of deployment pipeline(s) **1410** may use one or more of output models **1316** from training system **1304** and/or other models of applications to perform inference on imaging data. In at least one embodiment, two or more examples of inferencing using application orchestration system **1428** (e.g., a scheduler) may be available. In at least one embodiment, a first category may include a high priority/low latency path that may achieve higher service level agreements, such as for performing inference on urgent requests during an emergency, or for a radiologist during diagnosis. In at least one embodiment, a second category may include a standard priority path that may be used for requests that may be non-urgent or where analysis may be performed at a later time. In at least one embodiment, application orchestration system **1428** may distribute resources (e.g., services **1320** and/or hardware **1322**) based on priority paths for different inferencing tasks of AI services **1418**.

In at least one embodiment, shared storage may be mounted to AI services **1418** within system **1400**. In at least

one embodiment, shared storage may operate as a cache (or other storage device type) and may be used to process inference requests from applications. In at least one embodiment, when an inference request is submitted, a request may be received by a set of API instances of deployment system **1306**, and one or more instances may be selected (e.g., for best fit, for load balancing, etc.) to process a request. In at least one embodiment, to process a request, a request may be entered into a database, a machine learning model may be located from model registry **1324** if not already in a cache, a validation step may ensure appropriate machine learning model is loaded into a cache (e.g., shared storage), and/or a copy of a model may be saved to a cache. In at least one embodiment, a scheduler (e.g., of pipeline manager **1412**) may be used to launch an application that is referenced in a request if an application is not already running or if there are not enough instances of an application. In at least one embodiment, if an inference server is not already launched to execute a model, an inference server may be launched. Any number of inference servers may be launched per model. In at least one embodiment, in a pull model, in which inference servers are clustered, models may be cached whenever load balancing is advantageous. In at least one embodiment, inference servers may be statically loaded in corresponding, distributed servers.

In at least one embodiment, inferencing may be performed using an inference server that runs in a container. In at least one embodiment, an instance of an inference server may be associated with a model (and optionally a plurality of versions of a model). In at least one embodiment, if an instance of an inference server does not exist when a request to perform inference on a model is received, a new instance may be loaded. In at least one embodiment, when starting an inference server, a model may be passed to an inference server such that a same container may be used to serve different models so long as inference server is running as a different instance.

In at least one embodiment, during application execution, an inference request for a given application may be received, and a container (e.g., hosting an instance of an inference server) may be loaded (if not already), and a start procedure may be called. In at least one embodiment, pre-processing logic in a container may load, decode, and/or perform any additional pre-processing on incoming data (e.g., using a CPU(s) and/or GPU(s)). In at least one embodiment, once data is prepared for inference, a container may perform inference as necessary on data. In at least one embodiment, this may include a single inference call on one image (e.g., a hand X-ray), or may require inference on hundreds of images (e.g., a chest CT). In at least one embodiment, an application may summarize results before completing, which may include, without limitation, a single confidence score, pixel level-segmentation, voxel-level segmentation, generating a visualization, or generating text to summarize findings. In at least one embodiment, different models or applications may be assigned different priorities. For example, some models may have a real-time (TAT<1 min) priority while others may have lower priority (e.g., TAT<10 min). In at least one embodiment, model execution times may be measured from requesting institution or entity and may include partner network traversal time, as well as execution on an inference service.

In at least one embodiment, transfer of requests between services **1320** and inference applications may be hidden behind a software development kit (SDK), and robust transport may be provide through a queue. In at least one embodiment, a request will be placed in a queue via an API

for an individual application/tenant ID combination and an SDK will pull a request from a queue and give a request to an application. In at least one embodiment, a name of a queue may be provided in an environment from where an SDK will pick it up. In at least one embodiment, asynchronous communication through a queue may be useful as it may allow any instance of an application to pick up work as it becomes available. Results may be transferred back through a queue, to ensure no data is lost. In at least one embodiment, queues may also provide an ability to segment work, as highest priority work may go to a queue with most instances of an application connected to it, while lowest priority work may go to a queue with a single instance connected to it that processes tasks in an order received. In at least one embodiment, an application may run on a GPU-accelerated instance generated in cloud 1426, and an inference service may perform inferencing on a GPU.

In at least one embodiment, visualization services 1420 may be leveraged to generate visualizations for viewing outputs of applications and/or deployment pipeline(s) 1410. In at least one embodiment, GPUs 1422 may be leveraged by visualization services 1420 to generate visualizations. In at least one embodiment, rendering effects, such as ray-tracing, may be implemented by visualization services 1420 to generate higher quality visualizations. In at least one embodiment, visualizations may include, without limitation, 2D image renderings, 3D volume renderings, 3D volume reconstruction, 2D tomographic slices, virtual reality displays, augmented reality displays, etc. In at least one embodiment, virtualized environments may be used to generate a virtual interactive display or environment (e.g., a virtual environment) for interaction by users of a system (e.g., doctors, nurses, radiologists, etc.). In at least one embodiment, visualization services 1420 may include an internal visualizer, cinematics, and/or other rendering or image processing capabilities or functionality (e.g., ray tracing, rasterization, internal optics, etc.).

In at least one embodiment, hardware 1322 may include GPUs 1422, AI system 1424, cloud 1426, and/or any other hardware used for executing training system 1304 and/or deployment system 1306. In at least one embodiment, GPUs 1422 (e.g., NVIDIA's TESLA and/or QUADRO GPUs) may include any number of GPUs that may be used for executing processing tasks of compute services 1416, AI services 1418, visualization services 1420, other services, and/or any of features or functionality of software 1318. For example, with respect to AI services 1418, GPUs 1422 may be used to perform pre-processing on imaging data (or other data types used by machine learning models), post-processing on outputs of machine learning models, and/or to perform inferencing (e.g., to execute machine learning models). In at least one embodiment, cloud 1426, AI system 1424, and/or other components of system 1400 may use GPUs 1422. In at least one embodiment, cloud 1426 may include a GPU-optimized platform for deep learning tasks. In at least one embodiment, AI system 1424 may use GPUs, and cloud 1426—or at least a portion tasked with deep learning or inferencing—may be executed using one or more AI systems 1424. As such, although hardware 1322 is illustrated as discrete components, this is not intended to be limiting, and any components of hardware 1322 may be combined with, or leveraged by, any other components of hardware 1322.

In at least one embodiment, AI system 1424 may include a purpose-built computing system (e.g., a super-computer or an HPC) configured for inferencing, deep learning, machine learning, and/or other artificial intelligence tasks. In at least one embodiment, AI system 1424 (e.g., NVIDIA's DGX)

may include GPU-optimized software (e.g., a software stack) that may be executed using a plurality of GPUs 1422, in addition to CPUs, RAM, storage, and/or other components, features, or functionality. In at least one embodiment, one or more AI systems 1424 may be implemented in cloud 1426 (e.g., in a data center) for performing some or all of AI-based processing tasks of system 1400.

In at least one embodiment, cloud 1426 may include a GPU-accelerated infrastructure (e.g., NVIDIA's NGC) that may provide a GPU-optimized platform for executing processing tasks of system 1400. In at least one embodiment, cloud 1426 may include an AI system(s) 1424 for performing one or more of AI-based tasks of system 1400 (e.g., as a hardware abstraction and scaling platform). In at least one embodiment, cloud 1426 may integrate with application orchestration system 1428 leveraging multiple GPUs to enable seamless scaling and load balancing between and among applications and services 1320. In at least one embodiment, cloud 1426 may tasked with executing at least some of services 1320 of system 1400, including compute services 1416, AI services 1418, and/or visualization services 1420, as described herein. In at least one embodiment, cloud 1426 may perform small and large batch inference (e.g., executing NVIDIA's TENSOR RT), provide an accelerated parallel computing API and platform 1430 (e.g., NVIDIA's CUDA), execute application orchestration system 1428 (e.g., KUBERNETES), provide a graphics rendering API and platform (e.g., for ray-tracing, 2D graphics, 3D graphics, and/or other rendering techniques to produce higher quality cinematics), and/or may provide other functionality for system 1400.

FIG. 15A illustrates a data flow diagram for a process 1500 to train, retrain, or update a machine learning model, in accordance with at least one embodiment. In at least one embodiment, process 1500 may be executed using, as a non-limiting example, system 1400 of FIG. 14. In at least one embodiment, process 1500 may leverage services 1320 and/or hardware 1322 of system 1400, as described herein. In at least one embodiment, refined models 1512 generated by process 1500 may be executed by deployment system 1306 for one or more containerized applications in deployment pipelines 1410.

In at least one embodiment, model training 1314 may include retraining or updating an initial model 1504 (e.g., a pre-trained model) using new training data (e.g., new input data, such as customer dataset 1506, and/or new ground truth data associated with input data). In at least one embodiment, to retrain, or update, initial model 1504, output or loss layer(s) of initial model 1504 may be reset, or deleted, and/or replaced with an updated or new output or loss layer(s). In at least one embodiment, initial model 1504 may have previously fine-tuned parameters (e.g., weights and/or biases) that remain from prior training, so training or retraining 1314 may not take as long or require as much processing as training a model from scratch. In at least one embodiment, during model training 1314, by having reset or replaced output or loss layer(s) of initial model 1504, parameters may be updated and re-tuned for a new data set based on loss calculations associated with accuracy of output or loss layer(s) at generating predictions on new, customer dataset 1506 (e.g., image data 1308 of FIG. 13).

In at least one embodiment, pre-trained models 1406 may be stored in a data store, or registry (e.g., model registry 1324 of FIG. 13). In at least one embodiment, pre-trained models 1406 may have been trained, at least in part, at one or more facilities other than a facility executing process 1500. In at least one embodiment, to protect privacy and

rights of patients, subjects, or clients of different facilities, pre-trained models **1406** may have been trained, on-premise, using customer or patient data generated on-premise. In at least one embodiment, pre-trained models **1406** may be trained using cloud **1426** and/or other hardware **1322**, but confidential, privacy protected patient data may not be transferred to, used by, or accessible to any components of cloud **1426** (or other off premise hardware). In at least one embodiment, where a pre-trained model **1406** is trained at using patient data from more than one facility, pre-trained model **1406** may have been individually trained for each facility prior to being trained on patient or customer data from another facility. In at least one embodiment, such as where a customer or patient data has been released of privacy concerns (e.g., by waiver, for experimental use, etc.), or where a customer or patient data is included in a public data set, a customer or patient data from any number of facilities may be used to train pre-trained model **1406** on-premise and/or off premise, such as in a datacenter or other cloud computing infrastructure.

In at least one embodiment, when selecting applications for use in deployment pipelines **1410**, a user may also select machine learning models to be used for specific applications. In at least one embodiment, a user may not have a model for use, so a user may select a pre-trained model **1406** to use with an application. In at least one embodiment, pre-trained model **1406** may not be optimized for generating accurate results on customer dataset **1506** of a facility of a user (e.g., based on patient diversity, demographics, types of medical imaging devices used, etc.). In at least one embodiment, prior to deploying pre-trained model **1406** into deployment pipeline **1410** for use with an application(s), pre-trained model **1406** may be updated, retrained, and/or fine-tuned for use at a respective facility.

In at least one embodiment, a user may select pre-trained model **1406** that is to be updated, retrained, and/or fine-tuned, and pre-trained model **1406** may be referred to as initial model **1504** for training system **1304** within process **1500**. In at least one embodiment, customer dataset **1506** (e.g., imaging data, genomics data, sequencing data, or other data types generated by devices at a facility) may be used to perform model training **1314** (which may include, without limitation, transfer learning) on initial model **1504** to generate refined model **1512**. In at least one embodiment, ground truth data corresponding to customer dataset **1506** may be generated by training system **1304**. In at least one embodiment, ground truth data may be generated, at least in part, by clinicians, scientists, doctors, practitioners, at a facility (e.g., as labeled clinic data **1312** of FIG. **13**).

In at least one embodiment, AI-assisted annotation **1310** may be used in some examples to generate ground truth data. In at least one embodiment, AI-assisted annotation **1310** (e.g., implemented using an AI-assisted annotation SDK) may leverage machine learning models (e.g., neural networks) to generate suggested or predicted ground truth data for a customer dataset. In at least one embodiment, user **1510** may use annotation tools within a user interface (a graphical user interface (GUI)) on computing device **1508**.

In at least one embodiment, user **1510** may interact with a GUI via computing device **1508** to edit or fine-tune (auto) annotations. In at least one embodiment, a polygon editing feature may be used to move vertices of a polygon to more accurate or fine-tuned locations.

In at least one embodiment, once customer dataset **1506** has associated ground truth data, ground truth data (e.g., from AI-assisted annotation, manual labeling, etc.) may be used by during model training **1314** to generate refined

model **1512**. In at least one embodiment, customer dataset **1506** may be applied to initial model **1504** any number of times, and ground truth data may be used to update parameters of initial model **1504** until an acceptable level of accuracy is attained for refined model **1512**. In at least one embodiment, once refined model **1512** is generated, refined model **1512** may be deployed within one or more deployment pipelines **1410** at a facility for performing one or more processing tasks with respect to medical imaging data.

In at least one embodiment, refined model **1512** may be uploaded to pre-trained models **1406** in model registry **1324** to be selected by another facility. In at least one embodiment, this process may be completed at any number of facilities such that refined model **1512** may be further refined on new datasets any number of times to generate a more universal model.

FIG. **15B** is an example illustration of a client-server architecture **1532** to enhance annotation tools with pre-trained annotation models, in accordance with at least one embodiment. In at least one embodiment, AI-assisted annotation tools **1536** may be instantiated based on a client-server architecture **1532**. In at least one embodiment, annotation tools **1536** in imaging applications may aid radiologists, for example, identify organs and abnormalities. In at least one embodiment, imaging applications may include software tools that help user **1510** to identify, as a non-limiting example, a few extreme points on a particular organ of interest in raw images **1534** (e.g., in a 3D MRI or CT scan) and receive auto-annotated results for all 2D slices of a particular organ. In at least one embodiment, results may be stored in a data store as training data **1538** and used as (for example and without limitation) ground truth data for training. In at least one embodiment, when computing device **1508** sends extreme points for AI-assisted annotation **1310**, a deep learning model, for example, may receive this data as input and return inference results of a segmented organ or abnormality. In at least one embodiment, pre-instantiated annotation tools, such as AI-Assisted Annotation Tool **1536** in FIG. **15B**, may be enhanced by making API calls (e.g., API Call **1544**) to a server, such as an Annotation Assistant Server **1540** that may include a set of pre-trained models **1542** stored in an annotation model registry, for example. In at least one embodiment, an annotation model registry may store pre-trained models **1542** (e.g., machine learning models, such as deep learning models) that are pre-trained to perform AI-assisted annotation on a particular organ or abnormality. These models may be further updated by using training pipelines **1404**. In at least one embodiment, pre-installed annotation tools may be improved over time as new labeled clinic data **1312** is added.

Such components can be used to determine and optimize motion for an automated system to perform a determined task.

In at least some embodiments, aspects may be implemented or performed as set forth in the following clauses:

1. A method, comprising:

capturing motion data representing motion of a human hand interacting with an object in a physical environment;

analyzing the motion data using a physics simulator to generate a first representation of the motion of the human hand and a second representation of corresponding to motion of the object;

providing, based at least on the first representation and the second representation, a simulation of a virtual hand moving according to the first representation of motion and a virtual object moving according to the second

45

representation of motion in a virtual environment; detecting, during the simulation, simulated contact of a virtual robot with the virtual object;

deactivating, based at least on detecting the simulated contact, a controller driving the motion of the object in the simulation; and

causing the virtual hand to release a grasp of the virtual object in the virtual environment, wherein subsequent motion of the virtual object in the virtual environment will be dependent at least on operation of the virtual robot and a simulated gravity.

2. The method of clause 1, further comprising:

causing the simulated motion of the virtual object to be controlled independently of a simulated motion of the virtual hand in the virtual environment; and

ensuring, during the simulated motion, a realistic positioning of the virtual object relative to the virtual hand.

3. The method of clause 1, wherein the simulation includes simulating one or more collisions between the virtual hand and the virtual object.

4. The method of clause 1, further comprising:

simulating the gravity in the virtual environment, wherein the virtual object will be caused to fall after release of the virtual object by the virtual hand if the virtual robot does not properly grasp and hold the virtual object in the virtual environment.

5. The method of clause 1, further comprising:

comparing a simulated handover action performed by the virtual robot, in the virtual environment, against ground truth data for the handover action; and adjusting one or more parameters of a control model for the virtual robot based at least on the comparison.

6. The method of clause 1, further comprising:

performing the simulation for multiple models of the virtual hand with respect to multiple models of the virtual object, wherein the multiple models of the virtual object correspond to different types of objects.

7. The method of clause 1, wherein the motion of the human hand interacting with the object in the physical environment includes at least a motion of the human hand to grasp the object and move the object into a position and an orientation at which a handover is able to occur with respect to a gripper of a physical robot.

8. The method of clause 1, further comprising:

delaying the causing of the virtual hand to release the grasp of the virtual object until simulated contact of the virtual robot with the virtual object occurs for at least a threshold duration of time.

9. The method of clause 1, further comprising:

verifying that the simulated contact of the virtual robot with the virtual object corresponds to a permissible type of contact before causing of the virtual hand to release the grasp of the virtual object.

10. A method, comprising:

obtaining motion data representing related motion of a first object and a second object in a physical environment;

analyzing the motion data to generate one or more representations of the related motion;

providing, based at least on the one or more representations, a simulation of the related motion of the first object and the second object in a virtual environment;

detecting interaction of a virtual model of a robot with the first object in the virtual environment; and

modifying simulation of at least one of the first object or the second object in the virtual environment based at least on the interaction.

46

11. The method of clause 10, wherein the first object is a physical object, and the second object is a human hand holding the physical object, wherein the interaction of the virtual model of the robot with the first object in the virtual environment includes contact with the first object as part of a handover action.

12. The method of clause 11, wherein the motion data corresponds to at least a motion of the human hand to grasp the physical object and move the physical object into a position and an orientation at which the handover action is able to occur with respect to an appendage of the robot capable of receiving the object.

13. The method of clause 10, further comprising:

causing the simulated motion of the first object to be controlled independently of a simulated motion of the second object in the virtual environment; and

ensuring, during the simulated motion, a realistic positioning of the first object relative to the second hand.

14. The method of clause 10, wherein the simulation includes simulating one or more collisions between the first object and the second object.

15. The method of clause 10, further comprising:

simulating a force of gravity in the virtual environment, wherein the first object is able to be simulated to fall in case of a failed interaction of the virtual model of the robot with the first object.

16. A system, comprising:

one or more processing units to:

analyze motion data representing motion of a human hand interacting with an object in a physical environment to generate a first representation of the motion of the human hand and a second representation of corresponding motion of the object;

provide a simulation including a virtual robot to take possession of a virtual object from a virtual hand in a virtual environment;

cause, during the simulation, the virtual hand to move according to the first representation of motion and the virtual object to move according to the second representation of motion; and

determine a successful transfer of possession of the virtual object from the virtual hand to the virtual robot in response to the virtual object being in simulated contact with an end effector of the virtual robot for at least a minimum period of time.

17. The system of clause 16, wherein the first object is a physical object and the second object is a human hand holding the physical object, wherein the interaction of the virtual model of the robot with the first object in the virtual environment includes contact with the first object as part of a handover action.

18. The system of clause 17, wherein the motion data corresponds to at least a motion of the human hand to grasp the physical object and move the physical object into a position and an orientation at which the handover action is able to occur with respect to a gripper of the robot.

19. The system of clause 15, wherein the one or more processing units are further to:

cause the simulated motion of the first object to be controlled separately from a simulated motion of the second object in the virtual environment; and

ensure, during the simulated motion, a realistic positioning of the first object relative to the second hand.

20. The system of clause 15, wherein the system comprises at least one of:
 a system for performing simulation operations;
 a system for performing simulation operations to test or validate autonomous machine applications;
 a system for rendering graphical output;
 a system for performing deep learning operations;
 a system implemented using an edge device;
 a system incorporating one or more Virtual Machines (VMs);
 a system implemented at least partially in a data center; or
 a system implemented at least partially using cloud computing resources.

Other variations are within spirit of present disclosure. Thus, while disclosed techniques are susceptible to various modifications and alternative constructions, certain illustrated embodiments thereof are shown in drawings and have been described above in detail. It should be understood, however, that there is no intention to limit disclosure to specific form or forms disclosed, but on contrary, intention is to cover all modifications, alternative constructions, and equivalents falling within spirit and scope of disclosure, as defined in appended claims.

Use of terms “a” and “an” and “the” and similar referents in context of describing disclosed embodiments (especially in context of following claims) are to be construed to cover both singular and plural, unless otherwise indicated herein or clearly contradicted by context, and not as a definition of a term. Terms “comprising,” “having,” “including,” and “containing” are to be construed as open-ended terms (meaning “including, but not limited to,”) unless otherwise noted. Term “connected,” when unmodified and referring to physical connections, is to be construed as partly or wholly contained within, attached to, or joined together, even if there is something intervening. Recitation of ranges of values herein are merely intended to serve as a shorthand method of referring individually to each separate value falling within range, unless otherwise indicated herein and each separate value is incorporated into specification as if it were individually recited herein. Use of term “set” (e.g., “a set of items”) or “subset,” unless otherwise noted or contradicted by context, is to be construed as a nonempty collection comprising one or more members. Further, unless otherwise noted or contradicted by context, term “subset” of a corresponding set does not necessarily denote a proper subset of corresponding set, but subset and corresponding set may be equal.

Conjunctive language, such as phrases of form “at least one of A, B, and C,” or “at least one of A, B and C,” unless specifically stated otherwise or otherwise clearly contradicted by context, is otherwise understood with context as used in general to present that an item, term, etc., may be either A or B or C, or any nonempty subset of set of A and B and C. For instance, in illustrative example of a set having three members, conjunctive phrases “at least one of A, B, and C” and “at least one of A, B and C” refer to any of following sets: {A}, {B}, {C}, {A, B}, {A, C}, {B, C}, {A, B, C}. Thus, such conjunctive language is not generally intended to imply that certain embodiments require at least one of A, at least one of B, and at least one of C each to be present. In addition, unless otherwise noted or contradicted by context, term “plurality” indicates a state of being plural (e.g., “a plurality of items” indicates multiple items). A plurality is at least two items, but can be more when so indicated either explicitly or by context. Further, unless

stated otherwise or otherwise clear from context, phrase “based on” means “based at least in part on” and not “based solely on.”

Operations of processes described herein can be performed in any suitable order unless otherwise indicated herein or otherwise clearly contradicted by context. In at least one embodiment, a process such as those processes described herein (or variations and/or combinations thereof) is performed under control of one or more computer systems configured with executable instructions and is implemented as code (e.g., executable instructions, one or more computer programs or one or more applications) executing collectively on one or more processors, by hardware or combinations thereof. In at least one embodiment, code is stored on a computer-readable storage medium, for example, in form of a computer program comprising a plurality of instructions executable by one or more processors. In at least one embodiment, a computer-readable storage medium is a non-transitory computer-readable storage medium that excludes transitory signals (e.g., a propagating transient electric or electromagnetic transmission) but includes non-transitory data storage circuitry (e.g., buffers, cache, and queues) within transceivers of transitory signals. In at least one embodiment, code (e.g., executable code or source code) is stored on a set of one or more non-transitory computer-readable storage media having stored thereon executable instructions (or other memory to store executable instructions) that, when executed (i.e., as a result of being executed) by one or more processors of a computer system, cause computer system to perform operations described herein. A set of non-transitory computer-readable storage media, in at least one embodiment, comprises multiple non-transitory computer-readable storage media and one or more of individual non-transitory storage media of multiple non-transitory computer-readable storage media lack all of code while multiple non-transitory computer-readable storage media collectively store all of code. In at least one embodiment, executable instructions are executed such that different instructions are executed by different processors—for example, a non-transitory computer-readable storage medium store instructions and a main central processing unit (“CPU”) executes some of instructions while a graphics processing unit (“GPU”) executes other instructions. In at least one embodiment, different components of a computer system have separate processors and different processors execute different subsets of instructions.

Accordingly, in at least one embodiment, computer systems are configured to implement one or more services that singly or collectively perform operations of processes described herein and such computer systems are configured with applicable hardware and/or software that enable performance of operations. Further, a computer system that implements at least one embodiment of present disclosure is a single device and, in another embodiment, is a distributed computer system comprising multiple devices that operate differently such that distributed computer system performs operations described herein and such that a single device does not perform all operations.

Use of any and all examples, or exemplary language (e.g., “such as”) provided herein, is intended merely to better illuminate embodiments of disclosure and does not pose a limitation on scope of disclosure unless otherwise claimed. No language in specification should be construed as indicating any non-claimed element as essential to practice of disclosure.

All references, including publications, patent applications, and patents, cited herein are hereby incorporated by

reference to same extent as if each reference were individually and specifically indicated to be incorporated by reference and were set forth in its entirety herein.

In description and claims, terms “coupled” and “connected,” along with their derivatives, may be used. It should be understood that these terms may be not intended as synonyms for each other. Rather, in particular examples, “connected” or “coupled” may be used to indicate that two or more elements are in direct or indirect physical or electrical contact with each other. “Coupled” may also mean that two or more elements are not in direct contact with each other, but yet still co-operate or interact with each other.

Unless specifically stated otherwise, it may be appreciated that throughout specification terms such as “processing,” “computing,” “calculating,” “determining,” or like, refer to action and/or processes of a computer or computing system, or similar electronic computing device, that manipulate and/or transform data represented as physical, such as electronic, quantities within computing system’s registers and/or memories into other data similarly represented as physical quantities within computing system’s memories, registers or other such information storage, transmission or display devices.

In a similar manner, term “processor” may refer to any device or portion of a device that processes electronic data from registers and/or memory and transform that electronic data into other electronic data that may be stored in registers and/or memory. As non-limiting examples, “processor” may be a CPU or a GPU. A “computing platform” may comprise one or more processors. As used herein, “software” processes may include, for example, software and/or hardware entities that perform work over time, such as tasks, threads, and intelligent agents. Also, each process may refer to multiple processes, for carrying out instructions in sequence or in parallel, continuously or intermittently. Terms “system” and “method” are used herein interchangeably insofar as system may embody one or more methods and methods may be considered a system.

In present document, references may be made to obtaining, acquiring, receiving, or inputting analog or digital data into a subsystem, computer system, or computer-implemented machine. Obtaining, acquiring, receiving, or inputting analog and digital data can be accomplished in a variety of ways such as by receiving data as a parameter of a function call or a call to an application programming interface. In some implementations, process of obtaining, acquiring, receiving, or inputting analog or digital data can be accomplished by transferring data via a serial or parallel interface. In another implementation, process of obtaining, acquiring, receiving, or inputting analog or digital data can be accomplished by transferring data via a computer network from providing entity to acquiring entity. References may also be made to providing, outputting, transmitting, sending, or presenting analog or digital data. In various examples, process of providing, outputting, transmitting, sending, or presenting analog or digital data can be accomplished by transferring data as an input or output parameter of a function call, a parameter of an application programming interface or interprocess communication mechanism.

Although discussion above sets forth example implementations of described techniques, other architectures may be used to implement described functionality, and are intended to be within scope of this disclosure. Furthermore, although specific distributions of responsibilities are defined above for purposes of discussion, various functions and responsibilities might be distributed and divided in different ways, depending on circumstances.

Furthermore, although subject matter has been described in language specific to structural features and/or methodological acts, it is to be understood that subject matter claimed in appended claims is not necessarily limited to specific features or acts described. Rather, specific features and acts are disclosed as exemplary forms of implementing the claims.

What is claimed is:

1. A method, comprising:

capturing motion data representing motion of a human hand interacting with an object in a physical environment;

analyzing the motion data using a physics simulator to generate a first representation of the motion of the human hand and a second representation of a motion of the object;

providing, based at least on the first representation and the second representation, a simulation of a virtual hand moving according to the first representation of motion and a virtual object moving according to the second representation of motion in a virtual environment;

detecting, during the simulation, simulated contact of a virtual robot with the virtual object;

deactivating, based at least on detecting the simulated contact, a controller driving the motion of the object in the simulation; and

causing the virtual hand to release a grasp of the virtual object in the virtual environment, wherein subsequent motion of the virtual object in the virtual environment will be dependent at least on operation of the virtual robot and a simulated gravity.

2. The method of claim 1, further comprising:

causing the motion of the virtual object to be controlled independently of a simulated motion of the virtual hand in the virtual environment; and

ensuring, during the simulated motion, a realistic positioning of the virtual object relative to the virtual hand.

3. The method of claim 1, wherein the simulation includes simulating one or more collisions between the virtual hand and the virtual object.

4. The method of claim 1, further comprising:

simulating the gravity in the virtual environment, wherein the virtual object will be caused to fall after release of the virtual object by the virtual hand if the virtual robot does not properly grasp and hold the virtual object in the virtual environment.

5. The method of claim 1, further comprising:

comparing a simulated handover action performed by the virtual robot, in the virtual environment, against ground truth data for the handover action; and

adjusting one or more parameters of a control model for the virtual robot based at least on the comparing.

6. The method of claim 1, further comprising:

performing the simulation for multiple models of the virtual hand with respect to multiple models of the virtual object, wherein the multiple models of the virtual object correspond to different types of objects.

7. The method of claim 1, wherein the motion of the human hand interacting with the object in the physical environment includes at least a motion of the human hand to grasp the object and move the object into a position and an orientation at which a handover is able to occur with respect to a gripper of a physical robot.

8. The method of claim 1, further comprising:

delaying the causing of the virtual hand to release the grasp of the virtual object until simulated contact of the

51

virtual robot with the virtual object occurs for at least a threshold duration of time.

9. The method of claim 1, further comprising:

verifying that the simulated contact of the virtual robot with the virtual object corresponds to a permissible type of contact before causing of the virtual hand to release the grasp of the virtual object.

10. A method, comprising:

obtaining motion data representing related motion of a first object and a second object in a physical environment;

analyzing the motion data to generate one or more representations of the related motion;

providing, based at least on the one or more representations, a simulation of the related motion of the first object and the second object in a virtual environment; detecting interaction of a virtual model of a robot with the first object in the virtual environment; and

modifying simulation of at least one of the first object or the second object in the virtual environment based at least on the interaction.

11. The method of claim 10, wherein the first object is a physical object, and the second object is a human hand holding the physical object, wherein the interaction of the virtual model of the robot with the first object in the virtual environment includes contact with the first object as part of a handover action.

12. The method of claim 11, wherein the motion data corresponds to at least a motion of the human hand to grasp the physical object and move the physical object into a position and an orientation at which the handover action is able to occur with respect to an appendage of the robot capable of receiving the object.

13. The method of claim 10, further comprising:

causing the simulated motion of the first object to be controlled independently of a simulated motion of the second object in the virtual environment; and

ensuring, during the simulated motion, a realistic positioning of the first object relative to the second object.

14. The method of claim 10, wherein the simulation includes simulating one or more collisions between the first object and the second object.

15. The method of claim 10, further comprising:

simulating a force of gravity in the virtual environment, wherein the first object is able to be simulated to fall in case of a failed interaction of the virtual model of the robot with the first object.

52

16. A system, comprising:

one or more processors to:

analyze motion data representing motion of a human hand interacting with an object in a physical environment to generate a first representation of the motion of the human hand and a second representation of corresponding motion of the object;

provide a simulation including a virtual robot to take possession of a virtual object from a virtual hand in a virtual environment;

cause, during the simulation, the virtual hand to move according to the first representation of motion and the virtual object to move according to the second representation of motion; and

determine a successful transfer of possession of the virtual object from the virtual hand to the virtual robot in response to the virtual object being in simulated contact with an end effector of the virtual robot for at least a minimum period of time.

17. The system of claim 16, wherein the first object is a physical object and the transfer of possession of the virtual object from the virtual hand to the virtual robot is part of a handover action.

18. The system of claim 17, wherein the motion data corresponds to at least a motion of the human hand to grasp the physical object and move the physical object into a position and an orientation at which the handover action is able to occur with respect to a gripper of the robot.

19. The system of claim 16, wherein the one or more processors are further to:

cause a simulated motion of the virtual object to be controlled separately from a simulated motion of the virtual hand in the virtual environment; and ensure a realistic positioning of the virtual object relative to the virtual hand.

20. The system of claim 16, wherein the system is to:

perform simulation operations;

perform simulation operations to test or validate autonomous machine applications;

render graphical output;

perform deep learning operations;

use an edge device;

incorporate one or more Virtual Machines (VMs);

be implemented at least partially in a data center; or

be implemented at least partially using cloud computing resources.

* * * * *